Computers and Electrical Engineering 118 (2024) 109484

Contents lists available at ScienceDirect

Computers and Electrical Engineering

journal homepage: www.elsevier.com/locate/compeleceng

MP-GUARD: A novel multi-pronged intrusion detection and mitigation framework for scalable SD-IoT networks using cooperative monitoring, ensemble learning, and new P4-extracted feature set

Ameer El-Sayed [a], Wael Said [b], Amr Tolba [c,*], Yasser Alginahi [d], Ahmed A. Toony [e]

[a] Department of Information Technology, Faculty of Computers and Informatics, Zagazig University, Zagazig 44511, Egypt
[b] Computer Science Department, Faculty of Computers and Informatics, Zagazig University, Zagazig 44511, Egypt
[c] Computer Science Department, Community College, King Saud University, Riyadh 11437, Saudi Arabia
[d] Department of Computer Science, Adrian College, MI, USA
[e] The Egyptian Institute of the Alexandria Academy, Ministry of Higher Education and Scientific Research, Alexandria, Egypt

ARTICLE INFO

Keywords:
SD-IoT
Intrusion detection
Traffic monitoring
Attack mitigation
Feature Extraction
Multi-controller
Ensemble learning

ABSTRACT

The ever-increasing complexity of the Internet of Things (IoT) environment demands robust and adaptable intrusion detection frameworks, as existing approaches struggle with real-time traffic analysis, limited scalability, and static feature sets. This paper introduces MP-GUARD, a novel framework that leverages Software-Defined Networking (SDN), machine learning (ML), and a multi-controller architecture to address these challenges. MP-GUARD tackles multi-pronged intrusion attacks in IoT networks by offering real-time intrusion detection, collaborative traffic monitoring, and multi-layered attack mitigation. It achieves this through two core modules: P4-Assisted Cooperative Traffic Monitoring (CTM-P4) and Multi-Pronged Intrusion Detection and Mitigation (MPIDM). CTM-P4 facilitates real-time communication among multiple controllers, enabling dynamic feature extraction leveraging the interconnected state tables within P4-enabled switches. This module introduces a new 22-feature set (12 extracted and 10 computed) for comprehensive network analysis. MPIDM leverages the detailed network insights from CTM-P4 for attack identification and prevention. It introduces Stacked Ensemble Learning with Dynamic P4-Based Feature Selection (SELDP4-FS), achieving exceptional performance with 99.32 % accuracy, 99.24 % F1-score, and 0.49 % false positive rate. Additionally, MPIDM boasts efficient response and detection times of 16ms and 11ms, respectively. Beyond accuracy, MP-GUARD demonstrates significant advantages in terms of scalability and efficiency. The multi-controller architecture offers a 65 % reduction in overhead compared to single-controller setups. Furthermore, this work introduces the Mean Accuracy Steadiness Level (MASL) metric to assess model stability under varying traffic conditions. By combining P4-based feature extraction, dynamic feature selection, cooperative monitoring, ensemble learning, and a multi-controller architecture, MP-GUARD presents a significant contribution to IoT security, offering a scalable and adaptable solution for securing future SD-IoT deployments against evolving threats.

* Corresponding author.
E-mail address: atolba@ksu.edu.sa (A. Tolba).

https://doi.org/10.1016/j.compeleceng.2024.109484
Received 17 April 2024; Received in revised form 3 June 2024; Accepted 13 July 2024
Available online 23 July 2024
0045-7906/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

1. Introduction

The Internet of Things (IoT) has revolutionized various aspects of our lives, seamlessly connecting billions of devices and enabling a new era of data-driven applications [1]. However, this interconnectedness introduces significant security challenges. IoT devices are often resource-constrained, lack robust security protocols, and can become easy targets for malicious actors [2]. Intrusion detection systems (IDS) play a critical role in safeguarding IoT networks by identifying and mitigating unauthorized access attempts and malicious activities [3,4].

The landscape of IoT threats is constantly evolving, with attackers employing a diverse range of tactics [5]. These include Distributed Denial-of-Service (DDoS) attacks launched from botnets of compromised devices, side-channel attacks that exploit vulnerabilities in communication protocols, spoofing attacks that impersonate legitimate devices, flooding attacks that overwhelm networks with traffic, reconnaissance activities aimed at gathering sensitive information, web-based attacks targeting vulnerabilities in web interfaces, brute-force attacks attempting to guess credentials, stealthy attacks designed to evade detection, and dynamic attacks constantly adapting their tactics. The complexity and multi-pronged nature of these attacks necessitates robust and adaptable security solutions [6,7].

Moreover, attackers employ a diverse range of tactics that extend beyond the commonly known ones. These include Traffic Analysis Attacks, Anomaly Detection Attacks, Long-Lived Connections, Malformed Packets, Resource Utilization Analysis, Service Identification Attacks, Device-Specific Attacks, Location-Based Attacks, Scanning Attacks, Protocol-Specific Attacks, Botnet Formations, and Malware Propagation attacks [8]. In Traffic Analysis Attacks, attackers exploit bandwidth usage patterns to identify sensitive information or user behavior [9]. Anomaly Detection Attacks involve attempts to trigger anomaly detection systems with unusual traffic patterns to mask actual malicious activities [10]. Long-Lived Connections allow attackers to establish and maintain connections to exhaust network resources or maintain a foothold in the network [11]. Malformed Packets can be sent by attackers to exploit vulnerabilities in network devices or protocols. Resource Utilization Analysis enables attackers to analyze resource utilization patterns to identify critical infrastructure or areas with a weak security posture [12]. Service Identification Attacks involve using techniques to identify the types of services running on devices within the network to target vulnerabilities specific to those services. Device-Specific Attacks exploit known vulnerabilities in specific device models or operating systems commonly used in IoT deployments [13]. Location-Based Attacks leverage location information about devices to target geographically specific infrastructure or users. Scanning Attacks involve using automated tools to scan the network for vulnerabilities in devices or services. Protocol-Specific Attacks exploit weaknesses in specific communication protocols used by IoT devices [14]. Botnet Formations leverage compromised IoT devices to create botnets for launching DDoS attacks or other malicious activities. Malware Propagation involves using compromised IoT devices to spread malware to other devices within the network. These diverse attack vectors necessitate robust and adaptive security solutions that can not only identify traditional attacks but also adapt to emerging threats [15].

Software-Defined Networking (SDN) has emerged as a promising paradigm for managing and securing IoT networks. SDN decouples the control plane, responsible for network programmability and policy enforcement, from the data plane, which handles data forwarding. This separation offers several advantages for IoT security. First, SDN allows for the enforcement of consistent security policies across the entire network from a central controller, simplifying security management and configuration. Second, SDN empowers administrators to programmatically define network behavior using high-level languages, enabling the creation of dynamic and flexible security measures. Third, the control plane facilitates rapid detection and response to security incidents by enabling the dynamic reconfiguration of network policies [16].

SDN hinges on a control plane, typically a central controller, that manages network devices and data flow. When designing a secure and scalable network infrastructure for the IoT, the choice between a single-controller and a multi-controller architecture becomes crucial. While both approaches have their merits, understanding their differences is essential for optimal network security in SD-IoT deployments [17].

A single-controller architecture offers a centralized management approach. This centralized control simplifies initial setup and configuration. However, as the network scales with more devices and complex traffic patterns, this architecture faces limitations. A single controller can become a bottleneck, struggling to handle the increased traffic and management overhead. Furthermore, a single point of failure (SPOF) exists. If the central controller malfunctions, the entire network becomes inoperable, impacting availability [18].

In contrast, a multi-controller architecture distributes network management across multiple controllers. This distributed approach offers significant advantages. Firstly, the workload is spread across controllers, enabling the network to handle larger deployments and higher traffic volume. Secondly, the network exhibits enhanced fault tolerance. If one controller fails, the remaining controllers can take over its responsibilities, minimizing downtime and ensuring network continuity. Finally, distributing control tasks can improve performance by enabling faster decision-making closer to network devices, potentially reducing latency [19].

In the context of SD-IoT security, a multi-controller architecture offers distinct benefits. With a rapidly growing number of devices in an IoT network, a multi-controller architecture can efficiently manage this growth and the associated security demands. Even if one controller is compromised, the remaining controllers can continue to enforce security policies and detect intrusions, enhancing the overall security posture of the network. Additionally, security tasks like intrusion detection can be distributed across controllers, leading to a more responsive and robust security framework. In conclusion, a multi-controller architecture provides the necessary scalability, fault tolerance, and distributed security processing to effectively support security frameworks in complex and dynamic SD-IoT environments [20].

The P4 (Programming Protocol-Independent Packet Processors) programming language plays a crucial role in unlocking the full potential of SDN for intrusion detection in IoT networks [21]. P4 is a domain-specific language designed for programming the behavior

of programmable switches and network devices. It offers several advantages for intrusion detection. First, P4 programs can be deployed on switches, enabling intrusion detection to occur directly at the data plane, reducing latency and offloading processing from the central controller. Second, P4 provides access to a rich set of data fields within network packets, allowing for the extraction of features specifically tailored for intrusion detection. These features can complement traditional network traffic features and improve the accuracy of ML-based IDS models [22,23].

Machine learning (ML) has become an invaluable tool for intrusion detection. ML models can effectively analyze network traffic patterns and identify anomalies indicative of malicious activity [24]. Feature engineering plays a crucial role in ML-based IDS, as the selection and extraction of relevant features from network traffic significantly impact the model's accuracy and effectiveness [25]. This paper proposes a novel approach for intrusion detection in SD-IoT networks that leverages machine learning techniques.

For a clear understanding of our work, this paper is organized as follows. Section 2 presents the research contribution and novelty. Section 3 delves into existing research on intrusion detection for SD-IoT networks. Section 4 unveils the intricacies of our proposed methodology, MP-GUARD. Section 5 meticulously analyzes the experimental results and explores the framework's practical value. Section 6 discusses the limitations of our research and explores potential future directions. Finally, Section 7 concludes the paper by summarizing our findings.

2. Research contributions and novelty

This research is motivated by the need for a robust and scalable intrusion detection framework specifically designed for SD-IoT networks. Our proposed solution, MP-GUARD, addresses the limitations of existing approaches by leveraging the programmability and distributed nature of SDN. This section delves into the key contributions and novel aspects of the MP-GUARD framework, highlighting its innovative approach to intrusion detection in SD-IoT networks.

2.1. Research contributions

MP-GUARD introduces several advancements that enhance the effectiveness and efficiency of intrusion detection in SD-IoT environments. Here's a breakdown of its significant contributions:

- P4-based Feature Extraction with Dynamic Adaptation: A novel approach for real-time, dynamic feature extraction from network traffic is introduced. MP-GUARD leverages P4 programs running on switches to extract relevant features based on observed traffic patterns. This method utilizes match-action tables and state tables, resulting in a new feature set (12 extracted and 10 computed features) that complements traditional network traffic data, leading to improved intrusion detection accuracy.
- Cooperative Traffic Monitoring: A new cooperative traffic monitoring approach leverages the interconnected state tables within P4-enabled switches. This mechanism facilitates comprehensive data collection and analysis by enabling collaboration among distributed controllers. This collaborative approach aids in detecting complex attacks that might evade traditional methods.
- Ensemble Learning with Stacked Generalization and P4-Extracted Features: MP-GUARD employs an ensemble learning approach with stacked generalization. This technique integrates P4-extracted features alongside traditional network traffic features for training on IoT-specific datasets. This enriched feature set enhances the detection of multi-pronged attacks by providing a more comprehensive view of network traffic.
- Multi-Controller Architecture for Scalability and Fault Tolerance: The framework utilizes a multi-controller architecture to achieve superior scalability and fault tolerance compared to single-controller designs. This design distributes control plane functions across layers, enhancing network resilience and its ability to handle increased traffic loads in large-scale SD-IoT deployments.
- Dataplane-Level Intrusion Detection with P4 Integration: MP-GUARD incorporates a data plane-level intrusion detection framework operating on the data plane using P4 programs. This approach minimizes controller overhead and offloads the detection task from the controller, achieving early detection and mitigation. P4 programs enable real-time analysis of network traffic at the switch level, allowing for faster response times and reduced impact of malicious activities.

2.2. Novelty of MP-GUARD

MP-GUARD distinguishes itself from existing solutions through a groundbreaking combination of techniques specifically tailored for intrusion detection in SD-IoT networks. Here's a breakdown of the key aspects that contribute to MP-GUARD's novelty:

- Synergistic Technique Combination: The core innovation lies in the way MP-GUARD merges three distinct techniques: (1) P4-based feature extraction with dynamic selection for real-time feature extraction directly from network traffic. (2) Cooperative traffic monitoring that leverages interconnected P4 state tables for comprehensive data collection and analysis across distributed controllers. (3) Ensemble learning with stacked generalization, integrating P4-extracted features with traditional network traffic data for training machine learning models. While existing solutions might utilize some of these elements, MP-GUARD's unique application, particularly the use of P4 for dynamic feature extraction and state table-based monitoring, sets it apart.
- Mean Accuracy Steadiness Level (MASL): MP-GUARD introduces this novel metric (MASL) to assess the performance of machine learning models under fluctuating traffic patterns. MASL provides valuable insights into the robustness of these models in real-world scenarios with varying network loads.

- Superior Multi-Controller Architecture Performance: Our evaluation, based on a unique set of seven key performance indicators (KPIs), demonstrates the significant advantages of MP-GUARD's multi-controller architecture. This design demonstrably improves network throughput, reduces latency, and enhances packet processing speed. While there's a slight increase in controller CPU and memory utilization due to distributed processing, the substantial reduction in average overhead reduction rate (AORR) signifies a more efficient overall system. Notably, the decrease in packet delivery ratio (PDR) in the multi-controller approach indicates effective filtering of malicious traffic, leading to a cleaner network environment.

In essence, MP-GUARD's novel combination of techniques and the demonstrably superior performance of its multi-controller architecture position it as a significant advancement in securing complex SD-IoT networks.

3. Literature review

The ever-growing landscape of the Internet of Things (IoT) introduces new security challenges. Resource-constrained devices and a diverse attack surface make IoT networks vulnerable to various intrusion attempts. IDS plays a critical role in safeguarding these networks by identifying and mitigating malicious activities. Traditional methods and the rise of machine learning (ML) and deep learning (DL) approaches for intrusion detection in SD-IoT require careful consideration. Table 1 presents a summary of the reviewed related works.

3.1. Traditional intrusion detection methods Vs. ML and DL methods

Traditional intrusion detection methods include Signature-based IDS, Stateful Protocol Analysis (SPA), and Anomaly-based IDS. Signature-based IDS relies on pre-defined signatures of known attacks to detect malicious traffic [26,27]. Its strengths include simplicity and fast detection, but limitations include vulnerability to zero-day attacks and an inability to adapt to evolving threats [28]. Stateful Protocol Analysis inspects network traffic for adherence to established protocols, identifying deviations indicative of attacks [29]. Advantages include improved accuracy compared to signature-based methods, but limitations include high computational overhead and difficulty in handling novel protocols [30]. Anomaly-based IDS utilizes statistical models of normal network behavior to identify anomalies that may signify intrusions [31]. Strengths include the ability to detect unknown attacks, but limitations include high false positive rates and sensitivity to changes in network traffic patterns.

Table 1
Summary of intrusion detection techniques in SD-IoT networks.

Author(s)	Year	Research Methodology	Research Contribution	Research Gaps
Khedr et al. [29]	2023	Hybrid (ML, Stateful P4, Distributed Multi-Controller)	Integrates stateful processing and distributed architecture	Scalability in complex environments
Long et al. [36]	2022	Flow-based (Entropy & SSAE-SVM)	Improved accuracy over traditional entropy-based methods (98 %)	Limited effectiveness against multi-packet attacks
Zhang et al. [37]	2019	Flow-based (PSD based entropy & Machine Learning)	Detects low-rate DoS attacks	misbalancing accuracy and detection rates, vulnerable to multi-victim attacks
Xie et al. [38]	2019	Survey	Provides an overview of Machine Learning techniques in SDN	Lacks specific focus on intrusion detection
Zavrak & Iskefiyeli [39]	2023	Flow-based Anomaly Detection (RNNs & EncDecAD)	Good accuracy for unknown attacks (99.3 %)	OpenFlow overhead, limited by older dataset (ISCX2012)
Singh et al. [40]	2024	Knowledge & Data-driven Ensemble	Effective in emulated environments	Limited attack scenarios and rules for validation, data quality concerns
Ravi & Shalinie [41]	2020	Machine Learning (DDoS attack via SDN-cloud architecture)	Potential for DDoS attack detection and mitigation	Data mismatch between training data and real-world IoT traffic
Yin et al. [42]	2021	Machine Learning (SDN-IoT framework)	Improved accuracy for SD-IoT intrusion detection	Limited by underlying ML algorithms
Yousuf & Mir [43]	2022	Deep Learning (Recurrent Neural Network)	Detects DDoS attacks in IoT networks	Scalability challenges
Tang et al. [44]	2020	Deep Learning (Deep Intrusion Detection System)	Effective for DDoS detection with minimal network overhead	Requires optimization for accuracy and false positives
Yungaicela et al. [45]	2022	Deep Reinforcement Learning for low-rate DDoS in SDN	Mitigates slow-rate DDoS attacks	Scalability challenges for IoT networks.
Assis et al. [46]	2020	Deep Learning (Convolutional Neural Network)	Near real-time intrusion detection	Data compatibility with real-world IoT traffic complexities
Khan et al. [47]	2023	AutoML: Ensemble-based intrusion detection framework	High accuracy for different classifications.	Data limitations, neglects stateful scenarios
Awotunde et al. [48]	2021	Deep Learning (Feed-forward neural network)	Good accuracy in IIoT networks	Dataset not tailored for the IIoT domain
Aslam et al. [49]	2022	AMLSDM: An adaptive multilayered feed-forwarding scheme to identify DDoS attacks in SDN-based IoT networks	Detects DDoS attacks, focusing on analyzing the static features of the scrutinized network traffic.	The utilization of simulated traffic instead of real datasets obtained from IoT networks.

The rise of ML and DL techniques has revolutionized intrusion detection in SD-IoT networks. These approaches can learn from historical data to identify complex patterns and anomalies indicative of malicious activities. Supervised learning techniques like Support Vector Machines (SVM) and Random Forests (RF) have shown promising results in identifying specific attack types [32]. Strengths include high accuracy with proper training data, but limitations include vulnerability to adversarial attacks and the need for large, labeled datasets. Unsupervised learning techniques like K-Means clustering and Principal Component Analysis (PCA) can identify anomalies without labeled data. Strengths include applicability to unknown attacks, but limitations include difficulty in interpreting results and potential misidentification of normal traffic as anomalies. Deep learning, including deep neural networks (DNNs), has gained significant traction for intrusion detection due to their ability to learn complex features from network traffic data. Strengths include high accuracy and adaptability to diverse attack vectors, but limitations include high computational overhead, vulnerability to adversarial examples, and the requirement for even larger datasets compared to traditional ML techniques [33–35].

The choice between traditional and ML/DL methods depends on specific network requirements. Traditional methods offer simplicity and fast detection but struggle with adaptability, while ML/DL methods deliver high accuracy and adaptability but require significant computational resources and large training datasets. Additionally, the interpretability of results can be challenging, especially with deep learning models. In the next subsection, we will discuss recent related works for intrusion detection in SD-IoT.

3.2. SD-IoT intrusion detection related works

Securing SD-IoT networks from malicious actors is paramount due to their inherent vulnerabilities. Researchers have explored various intrusion detection methods to safeguard these systems. Initial approaches relied on statistical techniques. For instance, Long et al. [36] categorized traffic as normal or malicious based on packet payload entropy. However, these methods struggle with complex attacks that span multiple packets, rendering them susceptible to more sophisticated threats. Zhang et al. [37] proposed another flow-based method to differentiate normal and attack traffic using entropy thresholds. However, balancing accuracy and detection rates has proven challenging, often resulting in missed threats or false alarms.

Approaching from a different flow-based angle, Xie et al. [38] suggested leveraging rhythmic patterns of network traffic flows for intrusion detection. Although lightweight and resource-efficient, this approach may falter in high-traffic scenarios, leaving networks vulnerable at critical junctures. Moreover, when faced with sophisticated attacks concealing their traces within flow patterns, additional security measures are necessary.

Two recent studies explore flow-based anomaly detection methods that leverage the capabilities of Software-Defined Networking (SDN). The SAnDet system [39] takes advantage of SDN's programmability through a controller application. It utilizes Replicator Neural Networks (RNNs) and LSTM-based encoder-decoder (EncDecAD) methods to identify unknown attacks using flow features collected from OpenFlow switches. While achieving high accuracy, limitations include OpenFlow overhead, potentially hindering diverse IoT applications due to its static nature compared to P4-enabled switches, and the use of an older dataset (ISCX2012).

SecureFlow [40]proposes a framework for intrusion detection and dynamic rule configuration based on a knowledge-based and data-driven ensemble approach. It leverages the programmability of SDN for handling heterogeneous threats. SecureFlow exhibits good results in emulated environments; however, limitations include limited attack scenarios and rules for validation, requiring future work on real-world scalability and data quality considerations.

The emergence of Machine Learning (ML) and Deep Learning (DL) has introduced powerful tools for intrusion detection. LEDEM [41] demonstrates the potential of ML for identifying DDoS attacks in SDN-IoT networks. However, its reliance on a single classification method limits its adaptability to evolving attack vectors. Similarly, Yin et al. [42] present an ML-based architecture that achieves improved accuracy, but it's constrained by the limitations of the underlying ML algorithms.

Deep learning approaches show promise as well. DALCNN [43] utilizes Recurrent Neural Networks (RNNs) for DDoS attack detection, but faces challenges due to training data that may not accurately reflect the complexities of real-world IoT traffic. Tang et al.'s Deep Intrusion Detection System (DIDS) [44], based on Deep Neural Networks (DNNs), demonstrates promise for DDoS detection with minimal network overhead. However, it requires further optimization for enhanced accuracy and reduced false positives.

Recognizing the limitations of individual approaches, researchers have begun integrating diverse methodologies. For instance, a hybrid approach combining ML with stateful packet processing [29] utilizes a Double-Check Mapping Function to extract features before applying ML techniques for classification. While promising, this approach requires further exploration to tackle scalability challenges in complex SD-IoT environments. Deep Reinforcement Learning (DRL) [45] addresses low-rate DDoS attacks by leveraging DRL's adaptability, but scalability remains a hurdle.

Deep learning continues to evolve for intrusion detection. CNN-based intrusion prevention [46] aims to preemptively halt attacks at their source. However, ensuring compatibility between training data and real-world IoT traffic complexities is crucial for effectiveness. AutoML intrusion prevention [47] identifies potent supervised classifiers and constructs an ensemble method, boasting high accuracy. However, concerns remain about data set limitations and neglecting stateful scenarios, which are critical for comprehensive intrusion detection. Awotunde et al. [48] proposed a feed-forward neural network model for intrusion detection in IIoT networks, demonstrating good accuracy but with a dataset not tailored for the IIoT domain. Similarly, the AMLSDM architecture [49] focusing on analyzing static features of network traffic to detect DDoS attacks raises concerns about its effectiveness compared to more advanced techniques. Additionally, its use of simulated traffic instead of authentic datasets weakens its generalizability.

Despite the advancements, achieving a comprehensive shield for SD-IoT networks remains an ongoing pursuit. Key challenges include addressing data mismatch between training data and real-world network traffic, overcoming scalability bottlenecks to handle complex and large-scale networks, and effectively detecting multi-pronged attacks. Incorporating stateful considerations into

detection algorithms remains crucial for combating threats that exploit the intricate network connections within SD-IoT networks. Future research directions should focus on addressing these limitations to create a robust and adaptable intrusion detection system for SD-IoT networks.

4. The proposed MP-GUARD framework: a multi-layered approach to securing SD-IoT networks

MP-GUARD tackles security challenges in SD-IoT networks through a two-module approach, illustrated in Fig. 1. The first module, P4-Assisted Cooperative Traffic Monitoring (CTM-P4), focuses on collaborative traffic monitoring and dynamic feature extraction. CTM-P4 leverages P4-enabled switches to gather multifaceted features from network traffic. These features are then efficiently shared between distributed controllers, along with critical alerts, through collaboration between these controllers. This enables a comprehensive understanding of the network state.

The second module, Multi-Pronged Intrusion Detection and Mitigation (MPIDM), utilizes SELDP4-FS (Stacked Ensemble Learning with Dynamic P4-Based Feature Selection) for attack identification and prevention. SELDP4-FS benefits from the connected P4 state tables maintained by CTM-P4, providing a detailed picture of the network's current state. This allows for swift and accurate attack detection at the switch level, preventing the attack from reaching the controller and minimizing its impact. By combining multi-controller synchronization, cooperative monitoring, intrusion detection, and mitigation strategies, MP-GUARD's architecture design strengthens the overall network infrastructure against intrusions.

The MP-GUARD framework leverages a hierarchical multi-controller architecture to achieve superior scalability and fault tolerance within SD-IoT networks. This architecture distributes control plane functions across three layers: infrastructure, control, and data plane. The infrastructure layer houses the physical network elements, including potentially compromised IoT devices communicating with P4-enabled switches. These P4 switches are further categorized into P4-edge switches directly connected to the devices and P4-aggregator switches that collect and centralize statistical reports from the edge switches. The control plane, responsible for network management, is divided into domain controllers and a master controller. Domain controllers oversee specific network segments, while the master controller acts as the central authority, facilitating communication and ensuring synchronized operation across all domains. This hierarchical design offers significant advantages. Compared to single-controller architectures, the multi-controller approach demonstrably enhances scalability by distributing processing power across multiple controllers, as evidenced by the evaluation results showing up to a 42 % improvement in throughput and faster response times (up to 36 % reduction in latency). Additionally, the architecture fosters fault tolerance by ensuring that network functionality persists even if a single controller fails, due to the redundancy provided by the remaining operational controllers. Furthermore, the workload is distributed across controllers, preventing bottlenecks and improving overall network performance, especially under high-traffic conditions. This translates to smoother network operation and potentially faster response times to network requests.

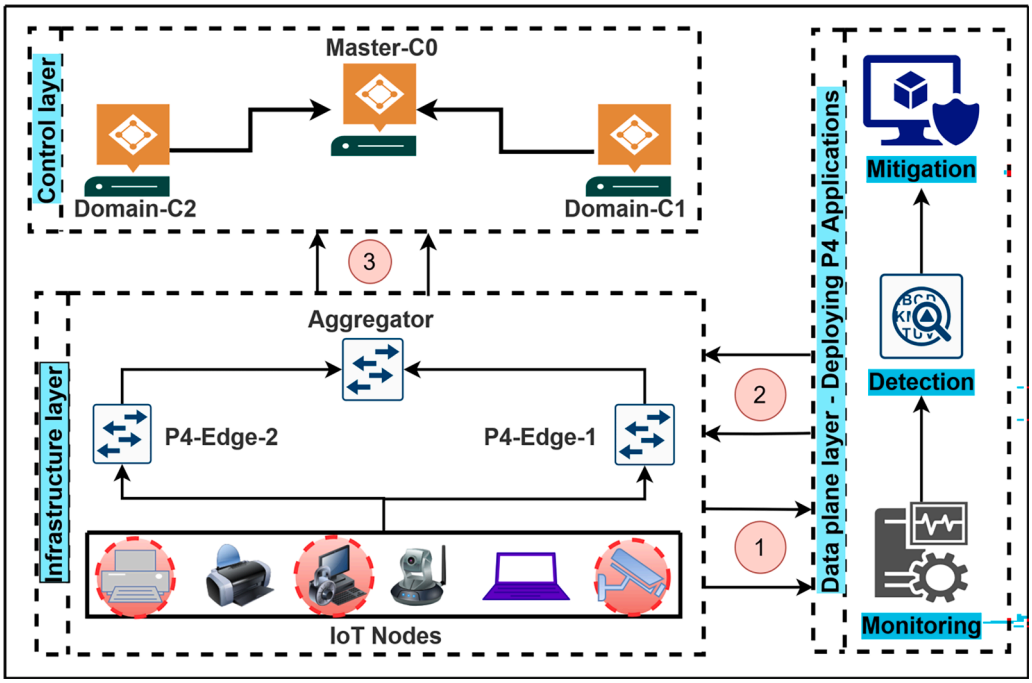


Fig. 1. The modular design of the proposed MP-GUARD framework.

4.1. The P4-assisted cooperative traffic monitoring (CTM-P4)Module

The P4-Assisted Cooperative Traffic Monitoring (CTM-P4) module leverages P4 state tables to dynamically collect traffic features from P4-enabled switches in an SD-IoT network. It fosters collaboration between distributed multi-connected controllers by efficiently sharing extracted features and critical alerts. The CTM-P4 algorithm presents a comprehensive feeding approach to the intrusion detection module, characterized by distinct phases and steps, listed in Algorithm 1. The Initialization Phase involves defining essential parameters, such as flow timeout and aging timers, which dictate the duration of flow information retention. Balancing the responsiveness to short-lived attacks with the detection of long-term malicious activity, the algorithm sets the foundation for subsequent processes. During this phase, the switch establishes vital connections with the intrusion detection module, context provider, and distributed SDN controllers. This connection enables data exchange, offering insights into device characteristics from the context provider and fostering collaborative decision-making among SDN controllers for effective attack mitigation and response.

Transitioning to the Packet Processing Phase, the algorithm engages in flow identification and matching when receiving packets. When a packet arrives, the switch extracts key elements to create a flow identifier, including source and destination IP addresses (SRCIP, DSTIP), protocol type (PROTP), source and destination ports, flow direction (FLDIR), input port (INPORT), and flow idle time (FLIDT). Notably, FLDIR indicates whether the flow is incoming (towards the device) or outgoing (from the device), aiding in detecting suspicious activity like unusual outbound sensor traffic or unauthorized inbound connections. Similarly, INPORT, the port where the packet enters the switch becomes valuable when combined with FLDIR. This allows for more granular traffic flow tracking and pinpointing potential issues within specific switch ports. Finally, including FLIDT in the identifier enables faster identification of potential denial-of-service attacks or long-lived connections that periodic calculations might miss. These key elements offer more granular flow tracking and improved intrusion detection capabilities. This flow identifier is then compared against existing entries in the P4 Match-Action table to identify associated flows. If a match is found, the corresponding P4 State table entry is accessed, providing

Algorithm 1
The proposed CTM-P4 algorithm.

01 Inputs: IoT packets, Flow timeout, Aging timers, and Context information.
02 Outputs: Flow statistics, Security alerts, Cooperative monitoring.

03 Phase 1: Initialization:
Define flow timeout and aging timers.
Initialize data structures for storing flow information (e.g., flow tables, counters).
Establish communication with the intrusion detection module.
Establish communication with the context provider.
Establish communication with all connected SDN controllers through a reliable communication protocol (gRPC).

09 Phase 2: Packet Processing:
For each (packet received on the switch) **do:**
    Extract flow key information (e.g., source IP, destination IP, protocol).
    Match the flow key information against entries in the P4 Match-Action table:
        If (a match is found):
            Access the corresponding P4 State table entry associated with the matched flow.
            Read the pre-defined counters and statistics (e.g., packet count, byte count, flow duration).
            Update these counters in the state table entry.
            Reset the flow timeout timer associated with the state table entry.
        End If
        If (no match is found):
            Create a new entry in the P4 Match-Action table with the extracted flow key information.
            Create a corresponding new entry in the associated P4 State table.
            Start the flow timeout timer for the newly created state table entry.
        End If
    Extract P4-specific features:
        Access header information (e.g., port numbers, TCP flags) directly from the received packet.
        Query switch resource utilization (e.g., buffer occupancy, queue length) through P4 APIs.
End For
For each (Window size == 2 seconds) **do:**
    Check the flow timeout timer for each entry in the P4 State table.
        If (entry timer == 0):
            Remove the corresponding entry from the P4 Match-Action table.
            Construct a message containing the accumulated flow statistics.
            Send the message to SDN controllers through the gRPC communication protocol.
        End If
End For

Phase 3: Collaboration between Controllers:
For each (packet received from the switch) **do**:
    Each controller processes the received flow statistics and context information.
    Controllers share the processed information with each other.
    Controllers use the gRPC protocol, enabling collaborative analysis and decision-making.
    Trigger mitigation actions on the relevant switches (e.g., traffic filtering, resource isolation) through P4.
    Send alerts to the security administrator for further investigation.
End For

direct access to pre-defined and readily calculated traffic statistics like packet count, byte count, and flow duration. Updates to these counters and the resetting of the flow timeout timer occur within the state table entry itself. If no existing match is found in the Match-Action table, a new flow entry is created in both the Match-Action table and the P4 State table. The state table entry is initialized with the necessary counters and statistics, and the flow timeout timer is started. A graphical scheme for the fully connected state table pipeline deployed in the CTM-P4 module is depicted in Fig. 2. In addition to flow-based features derived from the P4 state tables, the switch directly extracts P4-specific features such as header information (port numbers, TCP flags) and queries the switch's resources (buffer occupancy, queue length) via P4 APIs.

The Flow Timeout and Information Sharing Phase is crucial for both accurate flow table representation and efficient information sharing within our intrusion detection framework. To identify and remove expired entries, this phase employs a two-pronged approach. Each flow entry has a timer based on its duration (defined in Table 2) that helps identify inactive flows. Additionally, periodic scans proactively check all entries and remove those lacking recent updates. This redundancy ensures timely identification of expired entries, even for flows experiencing brief pauses exceeding the set duration.

Selecting the timeout interval for both the flow timer and periodic scans is critical. We consider factors like average flow duration, network dynamics, and security requirements. A configurable timeout interval allows for adjustments based on specific network characteristics, enabling network administrators to tailor the flow timeout mechanism to their needs. This comprehensive approach effectively identifies expired entries while maintaining the integrity of the flow table, contributing to the overall accuracy of the intrusion detection framework.

To complement the flow timeout timer's role in removing expired flows, MP-Guard utilizes a configurable aging timer. This timer focuses on managing flow table size and resource usage. It doesn't directly remove entries but can trigger actions like reducing the priority of older flows or marking them for potential removal when resources become constrained. The specific functionality and interaction with the flow timeout timer can be customized based on network requirements. The aging timer's initialization value is also configurable, allowing adjustments based on desired flow table size (preventing excessive growth) and network traffic volume (shorter

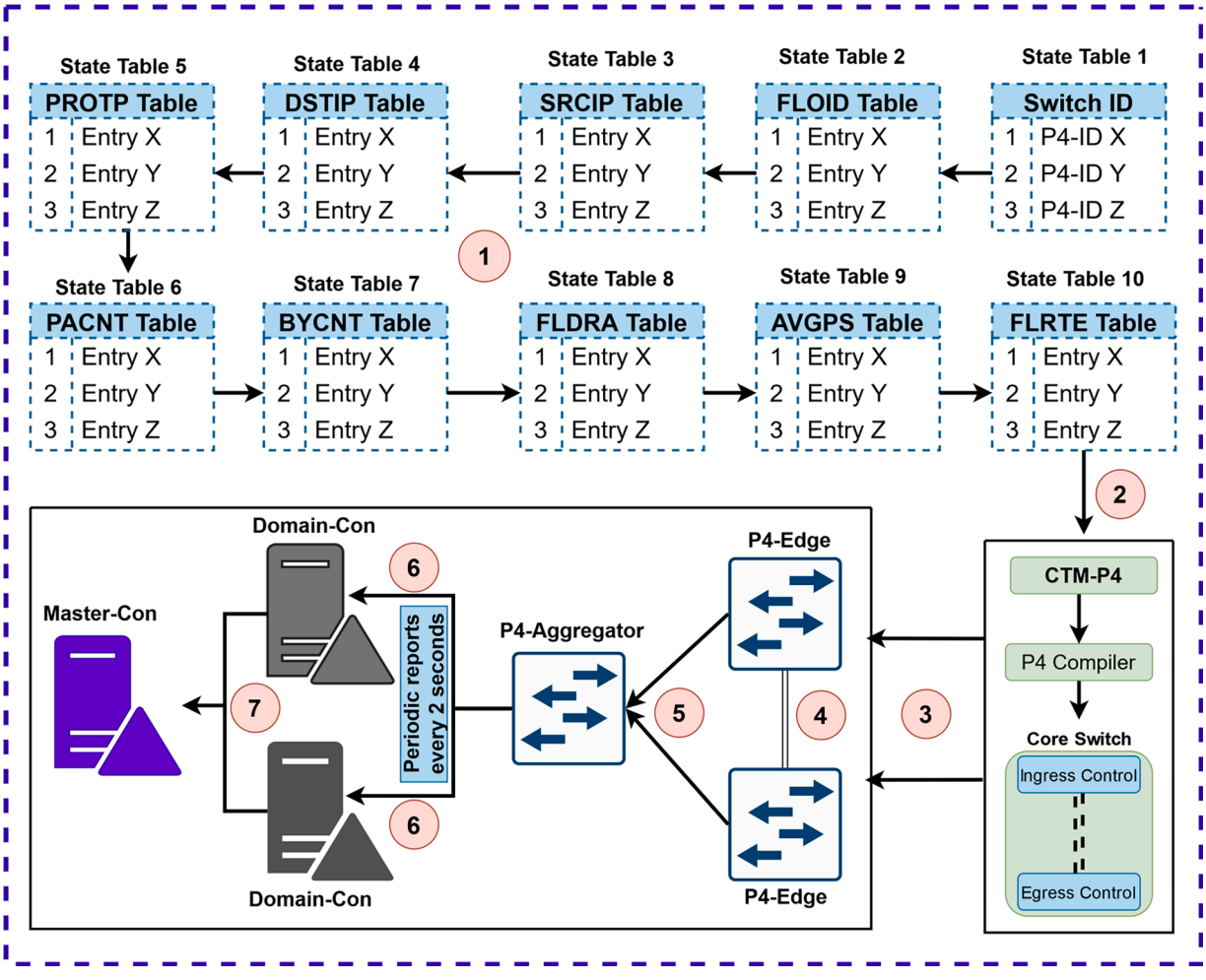


Fig. 2. Visual scheme of the CTM-P4 module implementation.

Table 2
Feature engineering summary: intrusion detection feature set in SD-IoT networks.

No	Abbr.	Feature Name	Feature Description	Extraction Approach	Extracted or Calculated	Feature Extraction Stage	Relevance to Attack Types	Feature Granularity Level
1	FLOID	Flow ID	Unique identifier for the traffic flow.	Assigned by switch	Extracted	Packet processing	All attacks	Flow-level
2	SRCIP	Source IP Address	The source IP address of the flow.	Extracted from the packet header	Extracted	Packet processing	Spoofing attacks, Source location identification	Flow-level
3	DSTIP	Destination IP Address	The destination IP address of the flow.	Extracted from the packet header	Extracted	Packet processing	Spoofing attacks, Destination identification	Flow-level
4	PROTP	Protocol Type	Network protocol used in the flow (e.g., TCP, UDP).	Extracted from the packet header	Extracted	Packet processing	Protocol-specific attacks, Anomaly detection	Flow-level
5	PACNT	Packet Count	Total number of packets in the flow.	Extracted using P4 counters	Extracted	Packet processing	DDoS, Botnet, Malware Propagation, Many Attacks	Global/Flow-level
6	BYCNT	Byte Count	Total number of bytes in the flow.	Extracted using P4 counters	Extracted	Packet processing	DDoS, Bandwidth analysis, Anomaly detection	Global/Flow-level
7	FLDRA	Flow Duration	Time elapsed since the flow started.	Calculated from flow entry timestamps	Calculated	Flow timeout	DoS attacks, Long-lived connections	Flow-level
8	AVGPS	Average Packet Size	The average size of packets in the flow.	calculated from Byte Count and Packet Count	Calculated	Packet timeout	Anomaly detection, Malformed packets	Flow-level
9	FLRTE	Flow Rate	The average rate of packets per second.	calculated from Packet Count and Flow Duration	Calculated	Packet timeout	DDoS attacks, Resource utilization analysis	Flow-level
10	HEAIF	Header Information	Specific fields extracted from packet headers (e.g., TCP flags, source/ destination port numbers).	Extracted from the packet header	Extracted	Packet processing	Protocol-specific attacks, Service identification	Flow-level
11	SWRUT	Switch Resource Utilization	Metrics indicating switch resource usage (e.g., buffer occupancy, queue length).	Accessed from switch APIs	Extracted	Periodically	Resource exhaustion attacks, Performance monitoring	Switch-level
12	DEVTP	IoT Device Type	Type of device generating the traffic (e.g., sensor, camera).	Retrieved from context provider	Extracted	Packet processing	Device-specific attacks, Anomaly detection	Flow-level
13	DEVLC	IoT Device Location	Physical location of the device (e.g., building floor, geographical coordinates).	Retrieved from context provider	Extracted	Packet processing	Location-based attacks, Anomaly detection	Device-level
14	IPAT	Inter-Packet Arrival Time	Time difference between consecutive packet arrivals in the flow.	Calculated from packet timestamps	Calculated	Packet processing	Spoofing attacks, Anomaly detection	Packet-level
15	FLENT	Flow Entropy	A measure of randomness in the flow's byte distribution.	Calculated from packet payloads	Calculated	Packet processing	Stealthy attacks, Anomaly detection	Flow-level
16	FLDCR	Flow Direction Change Rate	Rate of change in flow direction (incoming/ outgoing packets)	Calculated from flow statistics	Calculated	Packet processing	Scanning attacks, Port enumeration	Flow-level
17	NUDEP	Number of Unique Destination Ports	The number of distinct destination ports used in the flow.	Extracted from packet headers	Extracted	Packet processing	Port scanning attacks, Malicious connections	Flow-level
18	RTT	Round Trip Time	Time taken for a packet to reach the destination and return.	Calculated from timestamps in packets.	Calculated	Packet Processing	Anomalies, Network performance assessment	Flow-level
19	FLIDT	Flow Idle Time	The time between the last packet arrival and current time in the flow.	Calculated from flow statistics.	Calculated	Periodically	Denial-of-Service attacks, Long-lived connections	Flow-level
20	COSTR	Connection State Transitions	Tracks change in the connection state (e.g., SYN, SYN-ACK, ESTABLISHED).	Extracted from packet headers.	Extracted	Packet Processing	Scanning attacks, Port enumeration	Flow-level

(*continued on next page*)

Table 2 (*continued*)

No	Abbr.	Feature Name	Feature Description	Extraction Approach	Extracted or Calculated	Feature Extraction Stage	Relevance to Attack Types	Feature Granularity Level
21	PASEN	Entropy of Packet Sizes	The measure of randomness in the size distribution of packets in the flow.	Calculated from packet sizes.	Calculated	Packet Processing	Stealthy attacks, Anomaly detection	Flow-level
22	FLCOR	Flow Correlation	Analyzes relationships between different flows to identify potential coordinated attacks.	Network traffic analysis algorithms.	Calculated	Periodically	Multi-vector attacks, Advanced persistent threats	Global

timer for high-traffic environments).

Furthermore, the Flow Timeout and Information Sharing Phase optimizes resource allocation and information dissemination. When a flow entry expires, the switch generates a message containing accumulated flow statistics (as defined in Table 2) and additional context information from the context provider (e.g., device type, location). This message is then distributed to all connected SDN controllers. This approach offers two key advantages: 1) Resource Optimization: By promptly removing expired entries, the switch reduces storage overhead for the flow table, optimizing switch resources for handling active network traffic. 2) Real-Time Network State Awareness: The distribution of flow statistics and context information keeps all SDN controllers informed of the dynamic network state. This real-time awareness empowers controllers to make informed decisions regarding network traffic management, security enforcement, and overall network optimization.

At the Collaborative Analysis and Response stage, each SDN controller receives and processes flow statistics, fostering collaboration for a comprehensive analysis of network traffic behavior. Controllers share processed information, facilitating the detection of anomalies and potential attacks. Based on the combined analysis, controllers can coordinate actions to proactively mitigate threats. Controllers can send instructions to switches to enforce filtering, rate limiting, or resource isolation via P4, directly addressing suspicious behavior. In addition, security alerts can be generated and communicated to a security administrator, enabling timely intervention and investigation.

The algorithm boasts key advantages, including dynamic feature collection facilitated by P4 state tables, reducing the switch's processing burden. The distributed architecture with multi-connected controllers ensures scalability, accommodating the size and complexity of IoT networks. Offloading feature storage and updates to P4 state tables optimizes resource usage, enhancing overall efficiency. The algorithm's adaptability allows for dynamic adjustments based on evolving security requirements and the characteristics of the deployed IoT network. The CTM-P4 module stands as a robust and effective solution, leveraging P4 programmability, the distributed nature of SDN, and collaborative analysis for enhanced intrusion detection in SDN-based IoT networks.

The effectiveness of intrusion detection hinges on the ability to extract and analyze relevant features from network traffic. Table 2 provides a detailed overview of the feature set employed in our framework, categorized into two primary groups: extracted and calculated features. Table 2 summarizes these features, including their abbreviation, descriptive name, extraction approach, and relevance to various attack types. The "Feature Extraction Stage" column in Table 2 highlights the utilization of the P4 programming language for feature extraction. P4′s programmability allows us to directly extract features from network traffic at line speed, such as packet and byte counts, flow durations, and other flow-level statistics. This fine-grained access to network data enables the identification of features highly sensitive to specific attack behaviors. We complement this P4-driven feature extraction with insights from existing research on intrusion detection in IoT networks and SDNs, along with the expertise of security professionals. This multi-pronged approach ensures the chosen features effectively capture the fingerprints of various attacks targeting SD-IoT network environments.

Extracted features directly leverage readily available information from network traffic or switch resources. Packet headers offer a treasure trove of details, such as source and destination IP addresses, the communication protocol type, TCP flags utilized, and specific port numbers employed. These extracted features provide fundamental building blocks for anomaly detection and protocol-specific attack identification. Additionally, the P4 program maintains counters that track the total number of packets and bytes within a flow. These extracted counters offer valuable insights into traffic volume, potentially revealing anomalies like surges associated with DDoS attacks or malware propagation. Finally, our framework retrieves device type and location information from a dedicated context provider. By incorporating these extracted features, the system gains valuable context about the devices generating traffic, enabling tailored anomaly detection and the identification of location-based attacks.

Calculated features, on the other hand, delve deeper by extracting insights through analysis. Analyzing timestamps within packets allows for the calculation of flow duration, revealing the lifespan of a particular network flow. Similarly, calculating the inter-packet arrival time exposes potential inconsistencies that might indicate spoofing attempts. Statistical analysis of packet sizes within a flow yields the average packet size and its entropy, which can help detect malformed packets or attempts to mask malicious content. Flow statistics can be leveraged to calculate the flow direction change rate, aiding in the identification of suspicious activities like port scanning. Finally, by employing network traffic analysis algorithms, the system can calculate flow correlation, a powerful tool for uncovering coordinated attacks. These calculated features transform raw data points into valuable insights that empower the system to identify and respond to a wider range of intrusion attempts within the network. Each extracted/calculated feature plays a crucial role in identifying suspicious activities and constructing a comprehensive attack profile. These features offer valuable insights from diverse

perspectives, categorized into five key areas:

1. Basic Flow Information:
   The foundation of effective anomaly detection lies in fundamental flow details, encompassing Flow ID, Source/Destination IP, and Protocol. These features enable the identification of individual flows, allowing for the tracking of their origin, destination, and the employed communication protocol. This information proves invaluable in detecting spoofing attacks, analyzing traffic patterns, and identifying protocol-specific vulnerabilities. Additionally, analyzing the volume (Packet/Byte Count) and duration (Flow Duration) of traffic within a flow plays a key role in identifying potential anomalies. Significant increases in packet or byte count can point towards DDoS attacks or malware propagation, while abnormally long durations might suggest suspicious connections or long-lived exploits. Finally, fluctuations in Average Packet Size can reveal anomalies like malformed packets or attempts to evade network traffic analysis tools.
2. Advanced Flow Analysis:
   Moving beyond basic flow information, advanced features offer deeper insights into the characteristics of network traffic. Flow Rate plays a crucial role in detecting DDoS attacks aiming to overwhelm network resources with excessive traffic. By monitoring the rate of packets per second within a flow, the system can identify sudden surges in traffic volume, often characterizing DDoS attacks. Furthermore, Flow Entropy measures the randomness of the data within a flow, providing insights into the underlying data distribution. Significant deviations from expected entropy levels could indicate attempts to mask malicious content or evade detection by incorporating randomness into the data stream. Additionally, Flow Direction Change Rate (FLDCR) and Number of Unique Destination Ports (NUDEP) can help identify suspicious activities like port scanning or attempts to establish unauthorized connections from various ports. By monitoring these features, the system can detect rapid switching of communication directions or attempts to probe various ports, potentially searching for vulnerable entry points.
3. P4-Specific Features:
   Leveraging the programmability of SDN, P4-specific features offer valuable insights into network resource utilization and communication protocol details. Header Information, encompassing fields like TCP flags or source/destination port numbers, enables the identification of protocol-specific attacks and understanding of service usage patterns. Analyzing header information allows for a deeper understanding of the communication protocol employed and potential vulnerabilities associated with specific flag configurations or port usage. Moreover, Switch Resource Utilization (SWRUT) plays a crucial role in detecting resource exhaustion attacks aiming to overload the network infrastructure. By monitoring resource utilization metrics, the system can identify situations where attackers attempt to consume excessive resources, leading to potential network performance degradation or service disruptions. This information also aids in optimizing resource allocation and identifying potential performance bottlenecks, ensuring the efficient utilization of network resources.
4. Contextual Features:
   Incorporating context beyond network traffic analysis is crucial for identifying device-specific attacks and location-based anomalies. Device Type information allows for tailored anomaly detection and the identification of attacks targeting specific device types. For instance, unexpected traffic patterns from a sensor might indicate compromised device behavior. This feature helps leverage the specific characteristics and expected network usage patterns of different device types to identify deviations indicative of potential attacks. Additionally, Location information offers valuable context for understanding traffic patterns and identifying suspicious activities localized to specific network segments. This information can be particularly relevant for geographically distributed IoT networks, aiding in identifying location-based attacks or anomalies.
5. Packet-Level Analysis:
   Analyzing the time intervals between consecutive packets within a flow (Inter-Packet Arrival Time (IPAT)) helps detect spoofing attacks that attempt to mimic legitimate traffic patterns. Significant deviations in IPAT values might indicate suspicious activity, as legitimate traffic typically exhibits consistent patterns in packet arrival times. By analyzing the IPAT distribution, the system can identify inconsistencies that might point toward spoofing attempts. The combination of these diverse features within the framework's ensemble learning model facilitates a comprehensive attack detection strategy. By leveraging the unique insights offered by each feature and analyzing their combined impact, the system can effectively identify and respond to various and complex intrusion attempts within SDN-based IoT networks, ensuring the security and integrity of these critical infrastructures.

4.2. The multi-pronged intrusion detection and mitigation (MPIDM) module

This module, Multi-Pronged Intrusion Detection and Mitigation (MPIDM), utilizes SELDP4-FS (Stacked Ensemble Learning with Dynamic P4-Based Feature Selection) to act as the guardian of your IoT network. By leveraging the rich network state information provided by the connected P4 state tables in the CTM-P4 module, SELDP4-FS offers a comprehensive snapshot of network activity at the switch level. This empowers MPIDM to make real-time decisions, identifying and preventing attacks within the traffic flow itself, effectively stopping them before they reach the controller and potentially wreaking havoc on your network.

The cornerstone of MPIDM's swift and accurate detection lies in its strategic use of a 2-second window size (WS). This selection is not arbitrary but meticulously chosen based on the concept of Average Entropy Reduction Within a Window (AERW). To understand the rationale behind the 2-second window, we conducted a series of experiments evaluating window sizes ranging from 0.5 to 5.0 seconds. Each window size was assessed based on its effectiveness in capturing the entropy reduction caused by attacks (AERW). Our analysis revealed a fascinating sweet spot at the 2-second mark.

Here's why a 2-second window size reigns supreme: Early Attack Detection: Statistical significance tests confirmed that the 2-

second window captured the first and most crucial dip in entropy compared to normal traffic patterns. This translates to MPIDM's exceptional ability to swiftly identify the initial disruptions caused by attacks, preventing them from establishing a foothold in your network. Balancing Act: Efficiency vs. Accuracy: Window size selection involves a delicate balancing act between early detection and computational efficiency. Smaller windows (< 2 seconds) might theoretically provide even faster detection. However, they become more susceptible to noise and fluctuations inherent in normal network traffic, leading to potential false positives and wasted resources. Conversely, larger windows (> 2 seconds) could offer a seemingly more stable entropy reduction measurement, but at the cost of delayed detection. The 2-second window size strikes a perfect equilibrium, ensuring both timely attack identification and manageable computational demands for the system.

For a more granular understanding, Table 3 showcases the average drop rate (AVGDR) observed across ten different time-based window sizes. The AVGDR represents the decrease in entropy when comparing normal traffic patterns to those containing attacks within a specific window size. As evident from the table, the 2-second window exhibits a significant jump in AVGDR compared to smaller windows, indicating its superior ability to capture the entropy disruption caused by attacks. The AVGDR for a window size of (N) packets is calculated as shown in Eq. (1). In essence, the 2-second window size empowers MPIDM to achieve the best of both worlds: swift and accurate attack detection with minimal computational overhead. This translates to a more secure and responsive IoT network environment for your critical infrastructure.

$$AVGDR = \frac{WSNE - WSAE}{WSNE} \times 100 \tag{1}$$

4.2.1. The data preprocessing stage

The preprocessing stage is essential for creating a robust and accurate ML model for detecting intrusion attacks in SD-IoT. This stage involves several steps to ensure the accuracy and reliability of the ML model. The first step involves cleaning and filtering the data to remove any noise or irrelevant features. This is followed by feature selection, where only the most relevant features are retained to reduce the dimensionality of the data. The third step involves scaling the data to ensure that each feature is on the same scale and has equal weight in the model. To achieve this, we made use of the *sklearn.preprocessing* library's *StandardScaler* class. The resulting scaled features are calculated using Eq. (2), where "F" represents the new scaled feature, "f" is the initial feature, and "sd" is the standard deviation. Table 4 lists the training and testing details for each dataset.

$$F = \frac{f - mean\ (f)}{sd} \tag{2}$$

Table 3
Analyzing variability - average drop rates across ten different time-based window sizes.

WS	Attack Rate								AVGDR
		20 %	30 %	40 %	50 %	60 %	70 %	80 %	
0.5	WSNE	1.301	1.311	1.307	1.201	1.300	1.371	1.291	1.301
	WSAE	1.180	1.067	0.939	0.801	0.653	0.498	0.337	0.784
	WSDR	09.2 %	17.9 %	27.7 %	38.4 %	49.7 %	61.6 %	74.0 %	39.8 %
1.0	WSNE	1.380	1.380	1.380	1.380	1.380	1.380	1.380	1.382
	WSAE	1.243	1.123	0.987	0.840	0.685	0.522	0.353	0.834
	WSDR	09.8 %	18.6 %	28.4 %	39.0 %	50.3 %	62.1 %	74.3 %	40.6 %
1.5	WSNE	1.447	1.485	1.429	1.412	1.437	1.428	1.410	1.437
	WSAE	1.297	1.169	1.027	0.874	0.711	0.542	0.366	0.879
	WSDR	10.3 %	19.1 %	29.0 %	39.5 %	50.8 %	62.5 %	74.6 %	42.8 %
2.0	WSNE	1.564	1.566	1.569	1.569	1.570	1.575	1.579	1.571
	WSAE	1.343	1.110	1.062	0.903	0.823	0.801	0.378	0.862
	WSDR	11.13 %	16.29 %	29.42 %	40.9 %	51.15 %	63.79 %	79.84 %	46.8 %
2.5	WSNE	1.556	1.588	1.563	1.501	1.408	1.302	1.399	1.552
	WSAE	1.384	1.246	1.092	0.928	0.755	0.575	0.388	0.834
	WSDR	11.0 %	19.9 %	29.7 %	40.3 %	51.4 %	64.0 %	75.0 %	49.4 %
3.0	WSNE	1.602	1.605	1.406	1.229	1.291	1.009	1.391	1.608
	WSAE	1.421	1.278	1.120	0.951	0.773	0.589	0.397	0.987
	WSDR	11.2 %	20.2 %	30.0 %	40.6 %	51.6 %	64.2 %	75.1 %	52.7 %
3.5	WSNE	1.643	1.631	1.622	1.609	1.539	1.559	1.543	1.638
	WSAE	1.454	1.307	1.145	0.972	0.790	0.601	0.406	0.899
	WSDR	11.4 %	20.4 %	30.3 %	40.8 %	51.9 %	64.4 %	75.2 %	54.9 %
4.0	WSNE	1.681	1.682	1.667	1.601	1.591	1.410	1.581	1.676
	WSAE	1.484	1.333	1.167	0.972	0.805	0.612	0.413	0.969
	WSDR	11.6 %	20.6 %	30.5 %	40.8 %	52.0 %	64.5 %	75.3 %	57.0 %
4.5	WSNE	1.698	1.677	1.634	1.555	1.532	1.498	1.409	1.579
	WSAE	1.498	1.346	1.178	1.000	0.812	0.618	0.417	1.020
	WSDR	11.7 %	20.7 %	30.6 %	41.1 %	52.1 %	64.6 %	75.4 %	59.3 %
5.0	WSNE	2.0	2.101	2.003	2.112	2.301	1.997	1.926	2.023
	WSAE	1.739	1.556	1.359	1.150	0.933	0.708	0.477	1.159
	WSDR	13.0 %	22.1 %	32.0 %	42.4 %	53.3 %	64.5 %	76.1 %	61.9 %

Table 4
Data Samples and attack distribution in the ToN-IoT, Edge-IIoTset, and CICIoT2023 datasets.

Dataset	Attack Type	Total Samples	Training Samples	Testing Samples
Ton_IoT	Backdoor	506,118	103,794	34,598
	DoS	3,335,327	157,344	52,448
	DDoS	6,164,012	225,660	75,220
	Injection	489,457	84,744	28,248
	MITM	1040	789	263
	Scanning	6,120,194	177,762	59,254
	Ransomware	69,905	54,604	18,201
	Password	1,712,326	72,774	24,258
	XSS	2,078,729	57,774	19,258
	Normal	738,296	387,285	96,821
	Total	21,215,404	1,322,530	408,569
Edge-IIoTset	Backdoor	19,227	13,460	5767
	XSS	12,037	8426	3611
	DDoS-HTTP	37,349	26,144	11,205
	SQL-Injection	40,618	28,432	12,186
	MITM	276	193	83
	Vulner. Scan	39,187	27,439	11,757
	Port Scanning	15,957	11,169	4,788
	DDoS-UDP	96,459	67,521	28,938
	Ransomware	7649	5354	2295
	DDoS-TCP	40,032	28,022	12,010
	Password	38,759	27,131	11,628
	DDoS-ICMP	53,387	37,37	16,017
	Uploading	27,457	1922	8237
	Fingerprinting	638	446	192
	Normal	1,032,382	300,322	128,717
	Total	1,400,938	977,640	423,298
CICIoT2023	Backdoor	3216	2251	965
	XSS	3762	2633	1129
	DDoS-HTTP	28,735	20,114	8621
	SQL-Injection	5237	3666	1571
	ARP spoofing	307,527	194,622	64,874
	DNS spoofing	178,235	110,949	36,983
	Brute Force	13,034	9124	3910
	Vulner. Scan	37,376	26,163	11,213
	Port Scanning	82,280	57,696	24,584
	DDoS-UDP	5,412,267	288,587	123,684
	RSTFIN flood	4,044,348	248,044	106,304
	PSHACK flood	4,089,298	272,498	116,834
	Ping Sweep	2261	1583	678
	DDoS-TCP	4,432,178	302,525	129,653
	Comm. inject	5406	3784	1622
	DDoS-ICMP	7,200,487	140,34	60,147
	Uploading	1248	874	374
	UDP Fragment	286,457	200,523	85,937
	ACK Fragment	285,067	199,547	85,520
	ICMP Fragment	452,348	316,644	135,704
	SlowLoris	23,418	16,393	7025
	Total	5,894,482	4,125,182	1,769,300

Table 4 summarizes the data samples and attack distribution for three network intrusion detection datasets: ToN-IoT [50], Edge-IIoTset [51], and CICIoT2023 [52]. Each dataset includes various attack types, along with the total number of samples, training samples, and testing samples. Training samples are used to train intrusion detection models while testing samples are used to evaluate the model's performance on unseen data.

The table reveals some key differences between the datasets. Ton_IoT has the highest total number of samples (over 21 million), while Edge-IIoTset has a significantly lower number (around 1.4 million) and CICIoT2023 has the most (over 5.8 million). Ton_IoT also has a higher proportion of normal traffic compared to attack types, while CICIoT2023 has a larger number of samples for several specific attack types, including DDoS attacks (DDoS-UDP, RSTFIN flood, PSHACK flood, DDoS-TCP, and DDoS-ICMP). This suggests that CICIoT2023 might be more suitable for training models to detect these particular attacks. We implemented the following steps to execute the preprocessing stage:

- Step 1: A new label called *Traffic_Class* was created to classify all entries as normal or attack. For the binary classifier model, the *Traffic_Class* label includes the values 0 or 1, wherein attack traffic is represented by 1 and regular traffic by 0.
- Step 2: A new label called *Traffic_Type* was created to represent the attack classes.

- Step 3: The mean technique was used to identify and fix (or remove) faulty or incorrect entries from the dataset, such as duplication and missing data, such as NAN (Not A Number) and "INF" (Infinite Value).
- Step 4: Irrelevant flow parameters, such as timestamps and header data, were removed.
- Step 5: The *sklearn.model_selection* package's *train_test_split()* method was employed with a test size of 0.30 to indicate the size of the test set, and a random state of 42 was used to divide arrays into randomized train and test sets.
- Step 6: Categorical data was encoded using both the *LabelEncoder* and *OneHotEncoder* from the *sklearn.preprocessing* package. These classes convert categorical data into digits.
- Step 7: Independent variables were standardized using the *sklearn.preprocessing* library's *StandardScaler* class.
- Step 8: The initial features subset was obtained using the minimum redundancy maximum relevance (MRMR) method.
- Step 9: A second round of feature selection was performed using the neighborhood components analysis (NCA) approach on the feature subset obtained by MRMR to get the minimal-optimal subset and improve feature selection.
- Step 10: The best feature list was updated to include the chosen features.

4.2.2. P4-assisted feature engineering (AFE-P4)

P4 programming plays a crucial role in enabling efficient feature engineering for IDS within SDN-based IoT networks. This section delves into the workflow of a P4 program designed to extract relevant traffic features from both ingress and egress traffic, facilitating their utilization by the intrusion detection module. Fig. 3 focuses specifically on the workflow of the P4 program, while the sequence of operations within the MP-GUARD framework is illustrated in Fig. 4, including P4-based feature extraction. As shown in Fig. 3, the P4 program begins by defining the structure of packet headers using the parser. This allows for the extraction of specific fields relevant to feature engineering, such as source and destination IP addresses, protocol types, and port numbers. Conversely, the deparser is responsible for constructing outgoing packets based on specific header configurations. While not directly involved in feature extraction, it ensures proper packet manipulation based on the actions triggered by the program. The core of the feature extraction process lies in match-action tables. These tables define conditions (matches) based on specific header fields or other criteria. When a packet's header information matches the defined conditions, corresponding actions are triggered. Within the match-action tables, specific actions can be defined to extract and store relevant features. These actions can involve updating counters associated with flows or populating entries within state tables. P4 state tables offer a powerful mechanism for storing and managing flow-related information. These tables are dynamically created and updated based on the match-action table rules. When a packet triggers a match in the match-action table, the corresponding state table entry can be accessed. This entry can be used to store pre-defined counters for features like packet count, byte count, and flow duration, facilitating efficient feature extraction and accumulation.

The P4 program is compiled into a target-specific format using the P4 compiler. This compiled code can then be deployed on the P4-enabled switch, enabling it to understand and execute the defined functionalities. Fig. 5 provides a breakdown of the basic code structure of the intrusion detection application using P4, with comments to illustrate the blocks, while Fig. 6 presents the code blocks of

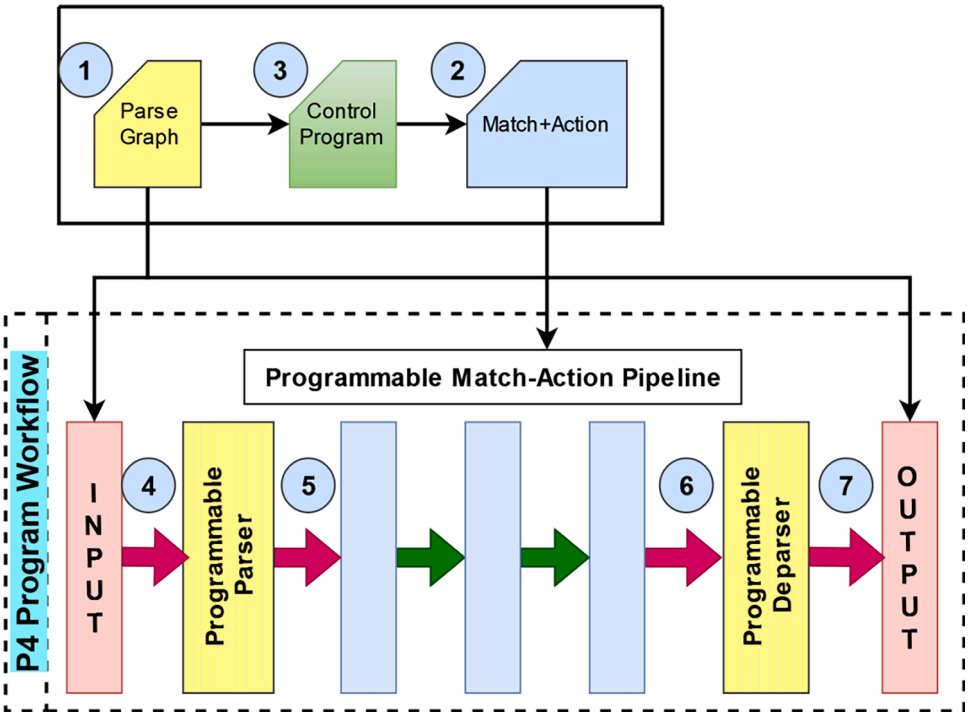


Fig. 3. P4 program workflow: a step-by-step breakdown.

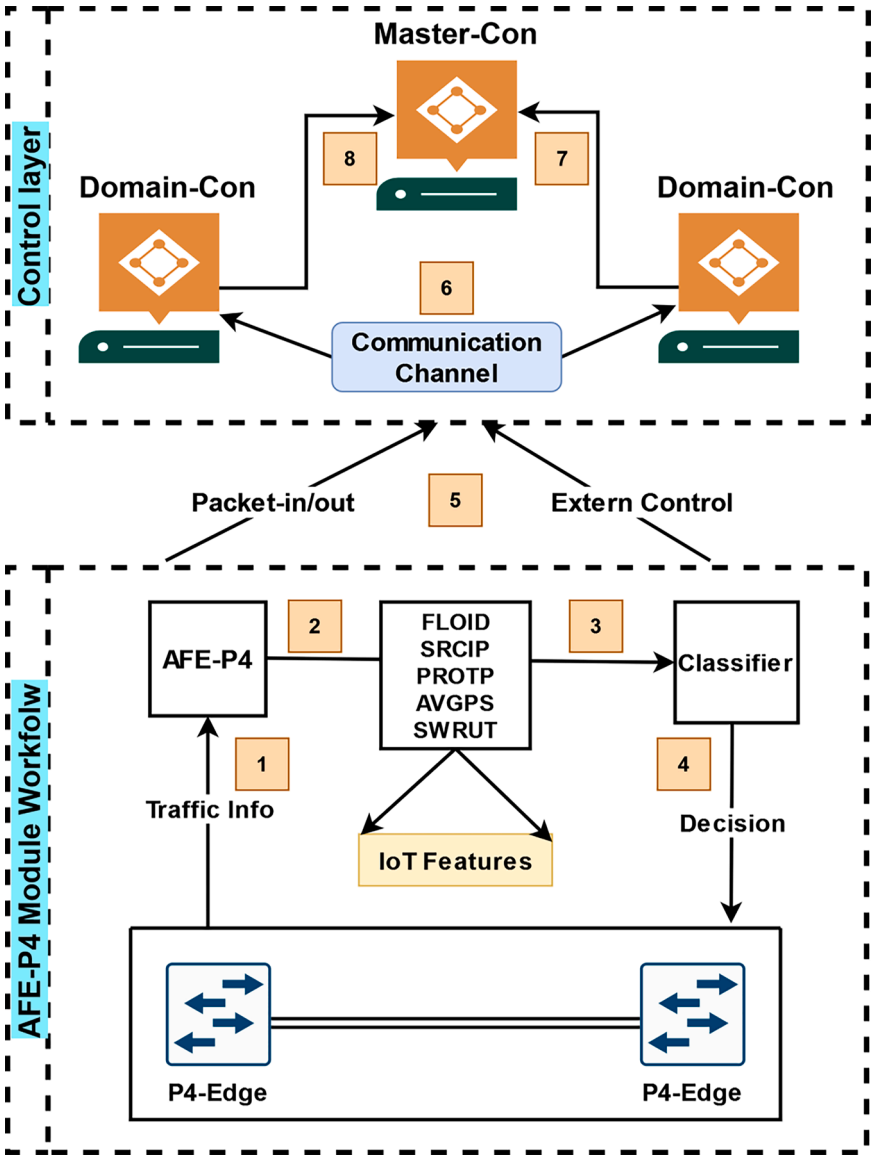


Fig. 4. Sequence of operations within the MP-GUARD framework.

the P4 feature extractor program. In Fig. 5, the program defines a header structure (MyHeader) containing relevant packet fields for matching. The *suspicious_traffic* table matches packets based on pre-defined criteria (e.g., source/destination IP, protocol). If a match occurs, the SET_SUSPICIOUS action is triggered, potentially marking the packet for further analysis or dropping it. The main table defines the overall control flow. Packets that don't match the suspicious traffic criteria are forwarded, while suspicious packets are dropped. Additionally, a meter can be incorporated for rate-limiting suspicious traffic.

In Fig. 6, the program utilizes a state table (*flow_tracker*) to keep track of ongoing flows. When a new flow is identified (based on source/destination IP), an entry is added with a unique flow ID. Subsequent packets belonging to the same flow update the corresponding entry with features like packet count and flow duration. The feature_extractor table processes packets based on the flow ID obtained from the metadata and updates the registers holding feature values. These extracted features can then be utilized by the intrusion detection program or exported for further analysis. In Fig. 6, the value of 1024 for flow entries represents a placeholder during initial implementation. MP-GUARD leverages P4's dynamic memory allocation mechanisms. Memory allocation for flow entries is determined at runtime based on network conditions and configured flow timeout/aging timers from the initialization phase. This ensures efficient memory usage while retaining crucial flow information for effective intrusion detection.

4.2.3. *Ml models selection stage*

The process of selecting base classifiers involves careful consideration of various factors. One critical aspect is the structure of the

```
// Define header fields used for matching
header MyHeader {
  bit srcIP;
  bit dstIP;
  bit protocol;
  // ... other relevant header fields
}

// Match table to identify suspicious traffic
table suspicious_traffic {
  key {
    MyHeader.srcIP;
    MyHeader.dstIP;
    MyHeader.protocol;
    // ... other matching criteria
  }
  actions {
    // Set a flag for further processing or packet drop
    SET_SUSPICIOUS;
  }
}
```

```
// Control flow for processing packets
table main {
  actions {
    // Forward allowed traffic
    FORWARD();
    // Drop suspicious traffic (based on previous table match)
    DROP();
  }

  meter my_rate_limiter {
    // Rate limiting configuration (optional)
  }

  selector {
    default: apply(suspicious_traffic);
    // Further match-action statements based on header fields
  }
}
```

Fig. 5. The intrusion detection P4 program code blocks.

```
// Define header fields and state tables
header MyHeader {
  // ... same as previous program
}

table flow_tracker {
  key {
    MyHeader.srcIP;
    MyHeader.dstIP;
    // Unique flow identifier
  }
  actions {
    ADD_FLOW(flow_id);  // Add flow to tracker
    UPDATE_FLOW(flow_id);  // Update flow state (
  }
  size: 1024;  // Allocate memory for flow entrie
}
```

```
// Extract features based on flow state
register packet_count;
register flow_duration;

meter my_flow_meter {
  // Track flow rate (optional)
}

table feature_extractor {
  actions {
    COUNT(packet_count);  // Increment packet count for flow
    UPDATE_DURATION(flow_duration);  // Update flow duration
  }

  selector {
    default: apply(flow_tracker);
    table_match {
      flow_tracker.flow_id == ingress_metadata.flow_meta;
      apply(feature_extractor);
    }
  }
```

Fig. 6. The feature extractor P4 program code blocks.

SD-IoT network, encompassing details such as the number and types of devices, network topology, and prevalent traffic patterns. Understanding the specific attacks to detect is equally crucial, as different classifiers may excel in identifying distinct attack types such as DoS, probing, or malware, each with its expected characteristics. Moreover, the available computational resources must be taken into account, considering factors like classifier complexity and the resource requirements within the IoT context. Additionally, the desired level of model interpretability plays a pivotal role in the decision-making process. Classifiers like Decision Trees (DT) and Bayesian Logistic Regression (BLR) offer more interpretable results, aiding in the comprehensive understanding of intricate attack patterns.

Among the considered classifiers are Random Forests (RF), known for their proficiency in handling high-dimensional data, capturing complex feature interactions, and providing robustness to noise and outliers. Decision Trees (DT) offer interpretable decision rules, enabling transparent reasoning; and facilitating real-time analysis in resource-constrained IoT devices. K-Nearest Neighbors (KNN) efficiently handle non-linear relationships, adapt to evolving attack patterns, and prove particularly effective for high-dimensional data and abnormal network behavior identification. Naive Bayes (GNB) is recognized for its computational efficiency and efficacy in handling imbalanced datasets, common in intrusion detection scenarios. Support Vector Machines (SVMs) are well-suited for high-dimensional feature spaces and excel in detecting outlier traffic patterns, making them effective for anomaly-based intrusion detection. Lastly, Bayesian Logistic Regression (BLR) combines logistic regression's interpretability with the ability to incorporate prior knowledge and handle uncertainty, proving effective in modeling probabilistic relationships between features and attack classes. In essence, the selection of base classifiers involves a comprehensive evaluation of their unique strengths in addressing

the intricacies of SD-IoT network dynamics and security challenges. Table 5 lists the Hyper-parameter configuration for the ML models. We have employed the Grid Search technique to explore a range of hyperparameter values and identify the configuration that yields the best performance on the used dataset

4.2.4. The intrusion detection and mitigation system

Traditional intrusion detection systems often struggle to keep pace with evolving attack methods and the dynamic environment of IoT deployments. This section introduces SELDP4-FS (Stacked Ensemble Learning with Dynamic P4-Based Feature Selection), a novel intrusion detection algorithm that leverages the strengths of P4 programming and stacked ensemble learning to address these challenges. The key innovation lies in the synergy between P4 and machine learning. P4 acts as a powerful feature extractor, dynamically collecting relevant information from network traffic through match-action and state tables. This extracted feature set encompasses not only traditional flow information (source IP, destination IP) but also P4-specific features (port numbers, TCP flags), and switch resource utilization data. This rich feature set provides a comprehensive view of network activity, enabling the intrusion detection system to make more informed decisions.

Algorithm 2 outlines the comprehensive process of SELDP4-FS, offering a robust solution for intrusion detection in SDN-IoT networks. This discussion breaks down the algorithm into distinct phases, explaining each step and its significance in achieving effective security. The algorithm begins by defining the set of base learners, diverse machine learning algorithms such as RF, SVM, GNB, DT, kNN, and BLR, to be employed in the stacked ensemble model. It then specifies the hyper-parameter configurations for each base learner, influencing their performance. The algorithm selects a Multi-Layer Perceptron (MLP) as the meta-learning model to combine the predictions from the base learners.

The first phase establishes the foundation for feature extraction by identifying initial relevant features based on prior security expertise and knowledge of common attack patterns. The P4 program then extracts various features from each network traffic flow, including flow-based information like source IP address, destination IP address, and protocol; P4-specific features like port numbers and TCP flags, providing insights into specific network traffic characteristics; and switch resource utilization like buffer occupancy and queue length, indicating potential resource strain caused by malicious activities.

The second phase focuses on designing the P4 program, which plays a crucial role in feature extraction and mitigation actions. Match-action tables are defined to extract chosen features from packet headers and network traffic based on specific match conditions. State tables are utilized to store flow-related information like packet count, byte count, and flow duration, enabling the system to track and analyze network traffic patterns over time. Finally, actions are defined within the match-action tables to perform feature extraction and update state table entries as needed.

The next phase emphasizes data preparation. The deployed P4 program dynamically extracts relevant features from live network traffic based on pre-defined match-action tables and state tables. Network traffic data encompassing various attack scenarios and normal network behavior is collected and labeled with corresponding attack types (e.g., Denial-of-Service (DoS), Man-in-the-Middle (MiTM)), or "normal" if no attack is present. This labeled data serves as the foundation for training the machine learning models.

Phase 4 involves training the machine learning models. Each base learner is trained on a portion of the labeled data to learn individual classification patterns. The MLP meta-learner is subsequently trained on the predictions generated by the base learners on

Table 5
Optimized hyperparameters for the employed machine learning models.

Model	Hyperparameter	Configuration
SELDP4-FS	Base classifier	RF, SVM, DT, BLR, kNN, GNB
	No. of base classifiers	6
	Number of estimators (RF)	150
	Number of trees (DT)	10
	Ensemble method	Weighted
RF	Split criteria	Gini
	Number of trees	150
	Minimum samples per leaf	1
	Minimum samples split	2
	Max features	4
DT	Split criteria	Gini
	Maximum depth	9
	Minimum samples per leaf	1
	Maximum features	4
KNN	Number of neighbors (N)	7
	Distance metric	Euclidean
	Weighting scheme	Uniform
SVM	Kernel	Linear
	Reg. parameter coefficient	10
BLR	Regularization parameter (C)	1
	Regularization	L2
	Solver	Sag
MLP	Number of neurons/ layer	22
	Activation function	Sigmoid
	No. of hidden layers	3

Algorithm 2
Stacked ensemble learning with dynamic P4-based feature selection (SELDP4-FS).

01 Initialization:
02 **Determine** the set of base learners (RF, SVM, GNB, DT, kNN, and BLR).
03 **Specify** base learners' hyper-parameter configurations.
04 **Develop** the meta-learning model using MLP.
05 Phase 1: Initial P4-Based Feature Extraction:
06 **For each** (IoT Traffic Flow):
07 **Identify** Initial relevant features based on prior knowledge**.**
08 **Extract** Flow-based information (source IP, destination IP, protocol).
09 **Extract** P4-specific features (port numbers, TCP flags).
10 **Extract** Switch resource utilization (buffer occupancy, queue length).
11 **End for**
12 Phase 2: P4 Program Design:
13 **Define** match-action tables to extract chosen features from packet headers and network traffic.
14 **Utilize** state tables to store flow-related information like packet count, byte count, and flow duration.
15 **Define** actions for feature extraction and state table updates within match-action tables.
16 Phase 3: Data Collection and Preprocessing:
17 **P4 program** dynamically extracts relevant features based on the defined match-action tables and state tables.
18 **Label** network traffic data, encompassing various attack scenarios and normal network behavior.
19 **Each** data point is labeled with the corresponding attack type (e.g., DDoS, MiTM) or "normal" if no attack is present.
20 Phase 4: Stacked Ensemble Learning Model Training:
21 **Each** base learner is trained on a portion of the labeled data (Portion 1).
22 **MLP** is trained on the predictions generated by the base learners on another portion of the labeled data (Portion 2).
23 Phase 5: Intrusion Detection:
24 **Deployed** P4 program continuously extracts features from network traffic in real-time**.**
25 **The extracted** features are fed into the trained stacked ensemble model.
26 **Based** on the model's prediction and a pre-defined threshold: Go to Phase 6
27 Phase 6: Multi-Layer Attack Mitigation:
28 **If** (attack is predicted)
29 **Apply** Traffic filtering to block malicious traffic flows.
30 **Adjust** filtering rules on P4 switches dynamically.
31 **Isolate** resources to prevent attackers from consuming network resources.
32 **Redirect** or drop malicious traffic flows at the P4 switches.
33 **Apply** rate limiting and traffic shaping in P4 switches.
34 **Dynamically** adjust security policies to minimize energy usage.
35 **Alerts** are sent to the security administrator for further investigation and potential response.
36 **End If**
37 Phase 7: Model Adaptation and Continuous Improvement:
38 **Use** metrics like detection accuracy, false positive rate, and response time to assess the model performance.
39 **Continuously** monitor the model performance in real-world network traffic.
40 **Update** P4 match-action tables to potentially include new features or modify existing ones.
41 **Periodically** retrain SELDP4-FS with newly labeled data to maintain effectiveness against emerging threats.

another portion of the labeled data. This training allows the meta-learner to combine the strengths of the individual learners and potentially achieve superior overall accuracy and generalizability.

Phases 5, 6, and 7 of the SELDP4-FS algorithm form the core of its real-world operation. These phases work together in a continuous cycle of detection, mitigation, and adaptation, ensuring the system's effectiveness in a dynamic and evolving network environment. In phase 5, the deployed P4 program acts as a vigilant sentinel, continuously extracting features from network traffic in real time. This ensures the system remains aware of any ongoing activity within the network. Extracted features are then fed into the pre-trained stacked ensemble model, built on the strengths of diverse base learners. The model analyzes the features and generates a prediction regarding the presence or absence of an attack. Based on the model's prediction and a pre-defined threshold, various mitigation actions are triggered through P4 if an attack is detected. These actions include traffic filtering, resource isolation, traffic flow control, and dynamic security policies.

Using traffic filtering, malicious traffic flows are blocked by dynamically adjusting filtering rules on P4 switches, preventing attackers from achieving their malicious goals and safeguarding network resources. In resource isolation, resources are isolated to further hinder attackers, preventing them from consuming excessive network resources that could disrupt legitimate traffic and network operations. Traffic flow control involves redirecting, dropping, or subjecting malicious traffic flows to rate limiting and traffic shaping within the P4 switches, effectively constraining the impact of attacks and protecting critical network resources. Dynamic

security policies allow for dynamically adjusting security policies on P4 switches to minimize energy usage while maintaining effective security measures, ensuring a balance between security and resource efficiency. Even when no attack is predicted, the system remains vigilant, sending alerts to the security administrator for any suspicious activity, allowing for further investigation and potential response if necessary.

In phase 7, the system continuously monitors its performance using metrics like detection accuracy, false positive rate, and response time. This ongoing evaluation helps identify potential weaknesses and areas for improvement. Furthermore, based on observed attack trends and evolving network conditions, P4 match-action tables can be dynamically updated, allowing the system to potentially include new features or modify existing ones, ensuring the feature set remains relevant and effective against emerging threats. To maintain effectiveness against evolving threats and adapt to network changes, the SELDP4-FS model is periodically retrained with newly labeled data. This retraining process incorporates the latest knowledge of attack methods and network behavior, ensuring the model's continued ability to accurately detect and respond to security threats. By continuously monitoring, analyzing, adapting, and improving, SELDP4-FS establishes a crucial cycle of responsiveness and resilience, ensuring the system remains proactive in its detection and mitigation efforts, and safeguarding SDN-IoT networks against the ever-changing landscape of cyber threats. Fig. 7 showcases the MP-GUARD framework's flowchart, emphasizing the mitigation process.

5. Performance measurement and analysis

5.1. MP-GUARD experimental setup

To comprehensively evaluate MP-GUARD's effectiveness against various attacks, a meticulously controlled experimental environment was created. This setup leveraged a virtual machine with an Intel Core i7-1360P CPU and 16GB RAM, running P4 version 16 enabled switches. Wireshark 2.2.0 captured network packets, while Xterm emulated attacker behavior through node interfaces.

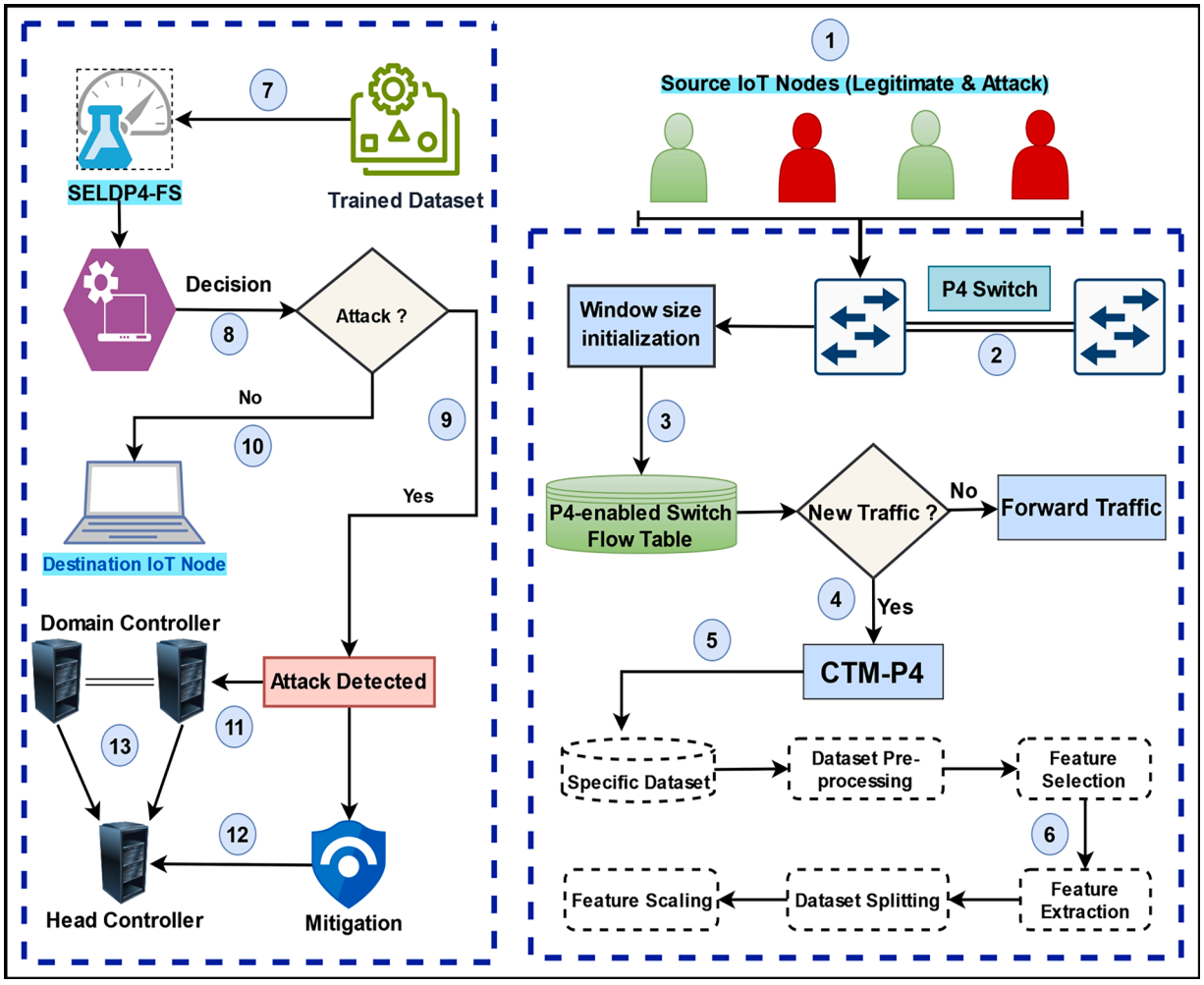


Fig. 7. Comprehensive flowchart of the proposed MP-GUARD framework.

Mininet-Wifi 2.2.0 and Python 3.7 scripts facilitated complex network configurations, and sFlow-RT monitored network activity.

The network itself was built on a hierarchical, multi-controller architecture. Seven P4 switches handled data processing, while network management was distributed across five controllers: one head controller, two domain controllers, and two backups. Communication between devices occurred through four access points within the virtualized environment. This intricate setup provided a realistic testing ground for MP-GUARD, allowing researchers to assess its ability to defend against diverse and multifaceted attacks before real-world deployment.

5.2. The MP-GUARD framework evaluation

To comprehensively evaluate MP-GUARD's effectiveness in securing SD-IoT networks, we employ various metrics based on True Positives (TP) - correctly identified attacks, True Negatives (TN) - accurately classified normal traffic, False Positives (FP) - mistakenly flagged normal events, and False Negatives (FN) - missed attacks. Table 6 details these metrics and their equations, while Fig. 8 (confusion matrices) visually represents the distribution of these classifications across different multi-class intrusion attacks. This multifaceted assessment ensures a thorough understanding of MP-GUARD's ability to detect and mitigate threats within the SD-IoT environment. Here's a breakdown of the key metrics used:

- Accuracy (ACC), Precision (PRE), F1-score (F1), and Recall (REC): These metrics provide a clear picture of how well MP-GUARD identifies actual attacks. Accuracy: Represents the comprehensive efficacy in precisely categorizing network traffic, encompassing both regular and malicious activities. It gauges the system's capability to correctly identify and differentiate between normal and attack-related data within the SD-IoT environment, providing a holistic measure of the intrusion detection system's performance. Precision measures the proportion of correctly identified attacks among all flagged events. A high precision indicates a low rate of false alarms (events flagged as malicious but actually normal traffic). Conversely, recall measures the proportion of actual attacks successfully detected by MP-GUARD. A high recall indicates a low rate of missed attacks (actual attacks that go unnoticed). F1-score balances these two concepts, providing a harmonic mean that considers both precision and recall.
- True Negative Rate (TNR) and Negative Predictive Value (NPV): These metrics focus on correctly classifying normal traffic. TNR or specificity measures the proportion of normal traffic correctly classified as normal. A high TNR indicates a low rate of misclassifying normal traffic as malicious. NPV goes a step further, measuring the proportion of events classified as normal that are actually normal. A high NPV indicates a low rate of missed normal traffic (normal traffic incorrectly classified as an attack).
- False Positive Rate (FPR), False Negative Rate (FNR), and False Discovery Rate (FDR): These metrics quantify the number of errors made by MP-GUARD in identifying attacks. FPR measures the proportion of normal traffic incorrectly identified as attacks. A low FPR is crucial to minimize unnecessary disruptions or resource exhaustion due to false alarms. FNR metric represents the proportion of actual attack instances that MP-GUARD fails to detect. It's crucial to minimize FNR to ensure a high level of security. A high FNR indicates that MP-GUARD is missing a significant number of attacks, leaving the network vulnerable. FDR takes a more specific look at flagged events, measuring the proportion of those that are actually false alarms. A low FDR indicates a reliable detection system with minimal false positives.
- Matthews Correlation Coefficient (MCC): This metric provides a balanced view of performance by considering all four instances (True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN)). It ranges from -1 (perfect disagreement) to +1 (perfect agreement). A high MCC indicates a well-calibrated system that effectively distinguishes between normal and attack traffic.
- Average Detection Time (ADT) and Mitigation Effectiveness Ratio (MER): These metrics assess the timeliness and effectiveness of MP-GUARD's response to attacks. ADT measures the average time it takes for MP-GUARD to detect an attack instance, calculated as the total detection time of all single attack instances (ΣDTSI) divided by the number of successfully detected instances (NSDAI). A low ADT indicates a fast response time to security threats. MER measures the proportion of successfully mitigated attack instances (NSMAI) compared to the total number of detected attacks (NSDAI). A high MER indicates the effectiveness of MP-GUARD in neutralizing identified threats.

Table 6
Evaluation metrics equations for MP-GUARD framework performance.

$T1 = (TP{*}TN)(3)$	$T2 = (FP{*}FN)(4)$
$T3 = (TP + FP)(5)$	$T4 = (TP + FN)(6)$
$T5 = (TN + FP)(7)$	$T6 = (TN + FN)(8)$
$ACC = \frac{TP+TN}{T4+T5}$ (9)	$PRE = \frac{TP}{T3}$ (10)
$REC = \frac{TP}{T4}$ (11)	$F1 = 2 * \frac{(PRE * REC)}{(PRE + REC)}$ (12)
$TNR = \frac{TN}{T5}$ (13)	$NPV = \frac{TN}{T6}$ (14)
$FPR = \frac{FP}{T5}$ (15)	$FDR = \frac{FP}{T3}$ (16)
$FNR = \frac{FN}{T4}$ (17)	$ADT = \frac{\sum_{i=1}^{n} DTSI}{NSDAI}$ (18)
$MER = \frac{NSMAI}{NSDAI} \times 100$ (19)	$MCC = \frac{(T1 - T2)}{\sqrt{((T3) * (T4) * (T5) * (T6))}}$ (20)

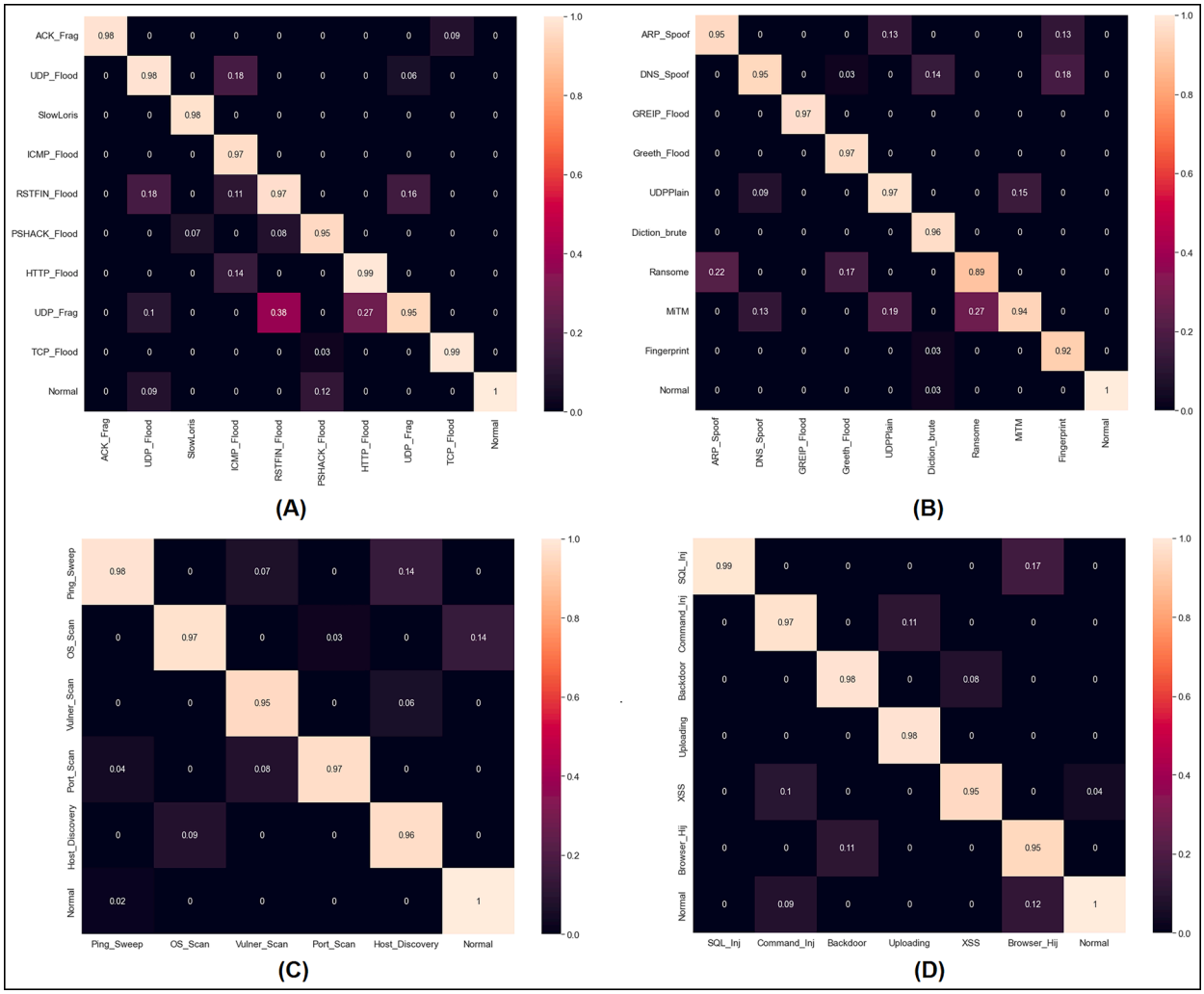


Fig. 8. Confusion matrices for multi-class attacks: (A) DDoS attacks, (B) different intrusions, (C) reconnaissance attacks, and (D) web-based attacks.

- Area Under the Curve (AUC): This metric analyzes the overall performance of the intrusion detection system. AUC evaluates the Receiver Operating Characteristic (ROC) curve, which plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various classification thresholds. The AUC measures the system's ability to distinguish between normal and attack traffic, ranging from 0.5 (random guessing) to 1 (perfect discrimination). A high AUC indicates a reliable system that can effectively differentiate between normal and malicious activities.
- Average Response Time (ART): This metric measures the average time it takes for MP-GUARD to take action (e.g., blocking traffic, sending alerts) after detecting an attack instance. It considers both detection time and mitigation execution time. A low ART reflects a prompt response to security incidents.

Table 7 presents the binary classification results of various machine learning models tested on three IoT datasets: TON_IoT, Edge-IIoTset, and CICIoT2023. The objective of this analysis is to evaluate the performance of SELDP4-FS compared to other models and identify the most suitable dataset for its application. A key observation emerges from the results, highlighting the overall superiority of SELDP4-FS. Across all three datasets, SELDP4-FS consistently outperforms other models, achieving the highest Accuracy, F1-score, Recall, Specificity, NPV, and AUC. This consistency underscores its effectiveness in accurately classifying normal and attack traffic, with a low false negative rate and a high true negative rate.

Starting with accuracy, SELDP4-FS consistently outperforms other models by a margin of at least 1.27 % (TON_IoT) and reaching as high as 3.09 % (CICIoT2023). Moving to the F1-score, SELDP4-FS maintains a lead across all datasets, ranging from 1.28 % (TON_IoT) to 1.77 % (Edge-IIoTset). This metric emphasizes a balance between precision and recall, highlighting SELDP4-FS's ability to achieve both accurate classifications and minimize errors. In terms of Recall, indicating the model's ability to identify true positive cases, SELDP4-FS achieves the highest values on all datasets. The margin ranges from 0.23 % (TON_IoT) to 1.85 % (CICIoT2023), demonstrating its effectiveness in minimizing false negatives. Similarly, SELDP4-FS exhibits the highest Specificity and NPV across all

Table 7
Binary classification results for various ML models on three IoT datasets.

Dataset	Model	Accuracy	Precision	F1-score	Recall	Specificity	NPV	AUC	MCC	FPR	FDR	FNR	ADT(ms)
TON_IoT	SELDP4-FS	94.72 %	94.15 %	94.43 %	94.65 %	94.75 %	94.30 %	94.82 %	91.48 %	5.28 %	5.85 %	5.70 %	21
	RF	93.46 %	92.85 %	93.15 %	93.42 %	93.55 %	93.05 %	93.96 %	89.95 %	6.49 %	7.15 %	6.95 %	29
	DT	92.43 %	91.72 %	92.07 %	92.39 %	92.55 %	92.05 %	92.03 %	88.46 %	7.42 %	8.28 %	8.05 %	36
	KNN	91.40 %	90.59 %	90.99 %	91.40 %	91.55 %	91.05 %	91.10 %	86.97 %	8.40 %	9.41 %	9.15 %	42
	GNB	90.15 %	89.34 %	89.74 %	90.25 %	90.45 %	89.95 %	90.25 %	85.43 %	9.75 %	10.66 %	10.45 %	50
	SVM	88.38 %	87.47 %	87.92 %	88.55 %	88.75 %	88.25 %	88.48 %	83.47 %	11.62 %	12.53 %	12.25 %	77
	BLR	86.11 %	85.60 %	85.85 %	86.43 %	86.65 %	86.15 %	86.51 %	81.43 %	13.75 %	14.40 %	14.15 %	75
Edge-IIoTset	SELDP4-FS	96.22 %	96.00 %	95.82 %	95.46 %	96.37 %	96.44 %	96.15 %	94.90 %	1.55 %	2.00 %	1.56 %	14
	RF	97.15 %	96.50 %	96.82 %	97.16 %	97.67 %	97.14 %	97.25 %	94.30 %	2.85 %	3.40 %	2.86 %	39
	DT	96.03 %	95.38 %	95.70 %	96.04 %	96.35 %	96.02 %	96.33 %	92.10 %	3.95 %	4.62 %	3.98 %	45
	KNN	95.11 %	94.26 %	94.68 %	95.12 %	95.43 %	95.10 %	95.41 %	90.18 %	4.89 %	5.64 %	4.90 %	56
	GNB	93.97 %	93.10 %	93.53 %	93.98 %	93.19 %	93.96 %	93.57 %	87.98 %	6.03 %	7.02 %	6.04 %	49
	SVM	92.23 %	91.18 %	91.70 %	92.24 %	92.55 %	92.22 %	92.63 %	84.28 %	7.75 %	8.82 %	7.78 %	64
	BLR	89.60 %	89.02 %	89.31 %	89.61 %	89.92 %	89.59 %	89.70 %	79.38 %	9.40 %	10.40 %	10.41 %	30
CICIoT2023	SELDP4-FS	99.32 %	99.77 %	99.23 %	98.59 %	98.50 %	98.87 %	99.52 %	99.68 %	0.49 %	0.75 %	2.13 %	11
	RF	97.89 %	98.42 %	97.88 %	98.40 %	98.61 %	97.35 %	99.07 %	96.33 %	0.75 %	1.00 %	2.65 %	32
	DT	97.56 %	98.17 %	97.55 %	98.21 %	98.72 %	96.83 %	98.59 %	95.40 %	1.00 %	1.25 %	3.17 %	59
	KNN	97.23 %	97.92 %	97.22 %	98.02 %	98.73 %	96.31 %	98.50 %	93.97 %	1.97 %	2.28 %	3.69 %	72
	GNB	96.90 %	97.67 %	96.89 %	97.83 %	97.34 %	95.79 %	97.52 %	92.42 %	2.16 %	2.63 %	4.21 %	95
	SVM	96.57 %	97.42 %	96.56 %	97.64 %	97.85 %	95.27 %	97.02 %	91.91 %	2.35 %	2.98 %	4.73 %	115
	BLR	96.24 %	97.17 %	96.23 %	97.45 %	97.96 %	94.75 %	96.54 %	89.82 %	2.54 %	3.33 %	5.25 %	66

datasets, translating to a low false positive rate. When considering the AUC, which provides a comprehensive overview of the model's performance across all classification thresholds, SELDP4-FS boasts the highest values on all datasets, signifying its strong overall classification ability. This dominance in core metrics positions SELDP4-FS as a robust intrusion detection system for SDN-IoT networks, capable of accurate attack classification with minimal errors.

An interesting observation arises from the varying performance of all models across datasets. SELDP4-FS reaches its peak performance (99.32 % Accuracy) on the CICIoT2023 dataset. This disparity can be attributed to the inherent characteristics of the datasets themselves. CICIoT2023 might be a more well-defined dataset with clearer distinctions between normal and attack traffic patterns, allowing the models to learn more effectively and leading to superior performance on this specific dataset. Fig. 9 shows a comparison of the three dataset performances in the binary classification, while Fig. 10 illustrates a comparison between ML models in terms of precision, recall, and F1-score for the CICIoT2023 dataset.

While showing generally strong performance, Random Forest (RF) consistently lags behind SELDP4-FS in most metrics across all datasets. The average difference in Accuracy between SELDP4-FS and RF is 1.33 %, highlighting the benefit of the ensemble learning approach employed by SELDP4-FS. Decision Tree (DT) follows a similar trend, with an average Accuracy of 1.97 % lower than SELDP4-FS. This suggests that the ensemble strategy helps overcome limitations inherent in decision tree structures. K-Nearest Neighbors (KNN), Gaussian Naive Bayes (GNB), Support Vector Machine (SVM), and Binomial Logistic Regression (BLR) exhibit a gradual decrease in performance compared to both SELDP4-FS and the tree-based models (RF, DT) on all datasets. The average Accuracy gap between SELDP4-FS and these models ranges from 2.91 % (KNN) to 6.97 % (BLR). This performance difference could be due to factors like limitations in handling complex data or overfitting the training data experienced by these models.

SELDP4-FS's prominent performance stems from a powerful combination of three key features: (1) Ensemble Learning that leverages multiple models for superior performance, (2) P4-Based Feature Extraction that dynamically captures both traditional and P4-specific flow information for richer analysis, and (3) Adaptive Feature Selection that allows the model to continuously adapt to evolving attack patterns and network conditions through P4 match-action table updates. Based on the results, CICIoT2023 appears to be the most suitable dataset for evaluating SELDP4-FS's peak performance, potentially containing well-defined and diverse attack scenarios that allow the model to learn effectively and achieve high accuracy. However, for real-world applications, it is crucial to consider the specific network environment where SELDP4-FS will be deployed.

Focusing solely on accuracy can be misleading when evaluating intrusion detection systems (IDS) for real-world deployment. Network environments are inherently dynamic, experiencing fluctuations in traffic volume and attack patterns. An IDS that performs well under controlled conditions might falter under the pressure of surging traffic or evolving attack strategies. This is why we introduced the Mean Accuracy Steadiness Level (MASL) as a novel evaluation metric. MASL goes beyond static accuracy by assessing a model's performance across a range of realistic traffic and attack intensities.

Our analysis using MASL revealed crucial insights. While models like BLR and SVM initially showed promising accuracy increases with rising flow rates, they displayed a concerning drop at higher attack intensities. For instance, BLR's accuracy peaked at 92.84 % (25,000 flow, 20 % attack) but then declined to 90.12 % (40,000 flow, 50 % attack). This suggests their vulnerability in high-traffic scenarios like DDoS attacks, where maintaining accuracy is paramount. Similarly, GNB (Gaussian Naive Bayes) and KNN (K-Nearest Neighbors) exhibited comparable patterns, with accuracy dropping from 91.72 % (10,000 flow, 10 % attack) to 88.34 % (40,000 flow, 50 % attack) for GNB and 90.45 % (10,000 flow, 10 % attack) to 86.12 % (40,000 flow, 50 % attack) for KNN, highlighting the limitations of these models in dynamic environments.

However, the MASL analysis also identified clear leaders. Decision Trees (DT) and Random Forests (RF) demonstrated consistent performance, with accuracy remaining relatively stable across various scenarios. DT's accuracy ranged from 94.17 % (10,000 flow, 10 % attack) to 93.89 % (45,000 flow, 35 % attack), while RF showed a range of 95.23 % (10,000 flow, 10 % attack) to 94.56 % (45,000

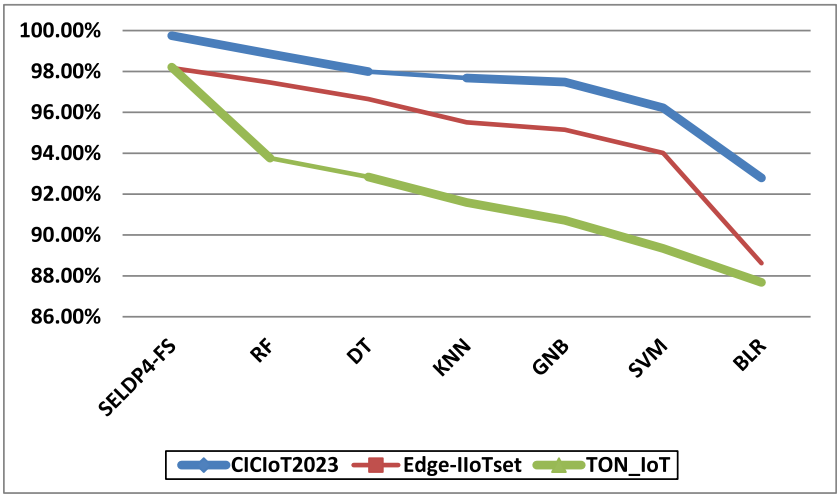


Fig. 9. Accuracy comparison across utilized datasets.

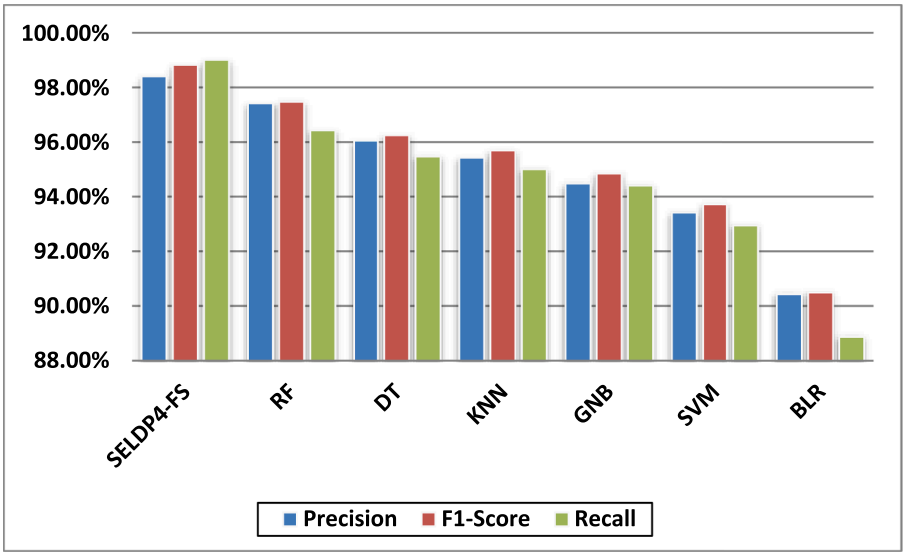


Fig. 10. Performance metrics comparison: precision, recall, and F1-score.

flow, 35 % attack). Yet, the true standout was the proposed SELDP4-FS model. It exhibited exceptional stability, showcasing a steady rise in accuracy as both traffic and attack rates increased. SELDP4-FS's accuracy grew from 97.39 % (10,000 flow, 10 % attack) to an impressive 99.45 % (45,000 flow, 35 % attack). This sustained high performance signifies SELDP4-FS's robustness and adaptability. Unlike models that might crumble under pressure, SELDP4-FS maintains its effectiveness even in high-traffic, high-attack situations.

The visualization in Fig. 11 further emphasizes the importance of MASL. It portrays the varying levels of stability among the models, highlighting the limitations of relying solely on peak accuracy. This analysis underscores the critical need to prioritize both accuracy and stability when selecting an IDS for real-world IoT deployments. By incorporating MASL alongside traditional accuracy metrics, we can ensure the selection of a solution that is not only effective but also adaptable to the dynamic threat landscape of the IoT.

Table 8 provides an evaluation of machine learning models' performance in detecting two common web-based attacks on IoT, leveraging the CICIoT2023 dataset. Notably, the SELDP4-FS model consistently exhibits superior performance across the two attack types. The SELDP4-FS algorithm introduces a unique approach to adaptive ensemble learning for intrusion detection in SD-IoT networks, showcasing features such as the "adaptive meta-ensemble," which adapts to evolving attack patterns by prioritizing reliable predictions based on real-time context. The meta-learner-based MLP integrates confidence scores from individual classifiers, enhancing ensemble decision-making by prioritizing contributions from more reliable models. Additionally, the "adaptive learning" feature allows SELDP4-FS to proactively adjust to changes in the threat landscape through continuous monitoring of performance metrics and dynamic relearning processes, enhancing its resilience against novel threats.

Examining specific attack types, SELDP4-FS consistently outperforms other models, as seen in its 98.57 % accuracy in detecting SQL injection, surpassing models like Random Forest (97.29 %) and Decision Trees (96.17 %). This demonstrates the efficacy of the

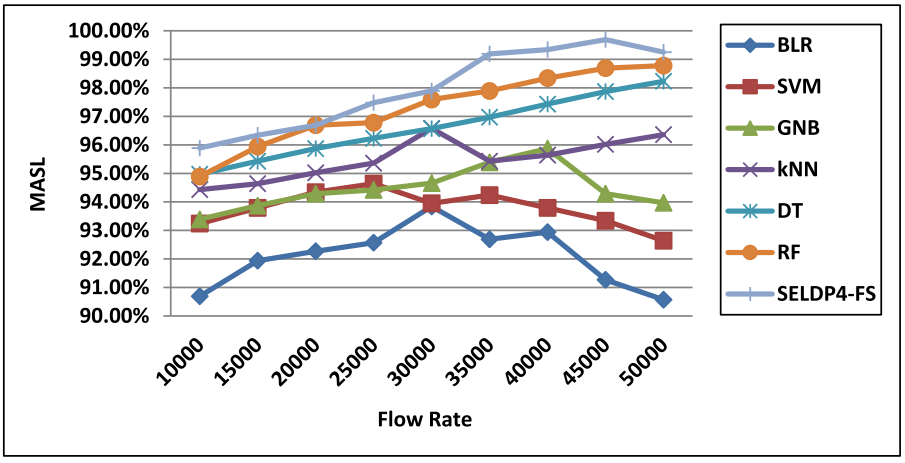


Fig. 11. The mean accuracy steadiness level (MASL) across tested ML models.

Table 8
Performance comparison of multi-class IoT web intrusion detection models on CICIoT2023 dataset.

Threat	Model	Accuracy	Precision	F1-score	NPV	Specificity	Recall	AUC	MCC	FPR	FDR	FNR	ADT(ms)
SQL injection	SELDP4-FS	98.57 %	98.02 %	98.29 %	98.69 %	98.83 %	98.30 %	98.15 %	97.14 %	1.30 %	1.98 %	1.70 %	16
	RF	97.29 %	96.71 %	97.00 %	97.12 %	97.43 %	97.45 %	97.09 %	94.28 %	2.87 %	3.39 %	2.55 %	22
	DT	96.17 %	95.43 %	95.80 %	95.98 %	95.59 %	96.33 %	96.27 %	92.17 %	4.01 %	4.74 %	3.67 %	34
	KNN	95.25 %	94.30 %	94.78 %	95.01 %	95.62 %	95.48 %	95.35 %	90.24 %	4.98 %	5.70 %	4.52 %	67
	GNB	94.03 %	93.11 %	93.57 %	93.77 %	93.88 %	94.28 %	94.43 %	88.06 %	6.22 %	7.22 %	5.72 %	117
	SVM	92.37 %	91.24 %	91.80 %	92.05 %	92.96 %	92.68 %	92.57 %	84.34 %	7.94 %	8.94 %	7.32 %	141
	BLR	89.72 %	89.04 %	89.38 %	89.69 %	89.10 %	89.15 %	89.62 %	79.43 %	9.30 %	10.30 %	10.85 %	123
Command injection	SELDP4-FS	97.40 %	98.05 %	97.70 %	98.15 %	98.25 %	97.25 %	98.72 %	90.55 %	1.85 %	1.72 %	2.75 %	25
	RF	97.05 %	97.70 %	97.35 %	97.90 %	97.50 %	96.80 %	98.42 %	94.43 %	2.10 %	2.05 %	3.20 %	45
	DT	96.70 %	97.35 %	97.00 %	97.65 %	97.75 %	96.35 %	98.21 %	93.41 %	2.35 %	2.40 %	3.65 %	65
	KNN	96.35 %	97.00 %	96.65 %	97.40 %	97.80 %	95.90 %	97.82 %	91.92 %	2.60 %	2.75 %	4.10 %	87
	GNB	96.00 %	96.65 %	96.30 %	97.15 %	97.25 %	95.45 %	97.57 %	90.76 %	2.85 %	3.10 %	4.55 %	84
	SVM	95.65 %	96.30 %	95.95 %	96.90 %	96.30 %	95.00 %	97.26 %	89.85 %	3.10 %	3.45 %	5.00 %	124
	BLR	95.30 %	95.95 %	95.60 %	96.65 %	96.95 %	94.55 %	96.91 %	88.43 %	3.35 %	3.80 %	5.45 %	92

Table 9
Performance comparison of multi-class IoT flooding intrusion detection models on CICIoT2023 dataset.

Threat	Model	Accuracy	Precision	F1-score	NPV	Specificity	Recall	AUC	MCC	FPR	FDR	FNR	ADT(ms)
RSTFIN flood	SELDP4-FS	97.12 %	96.53 %	96.82 %	96.97 %	97.01 %	96.90 %	97.72 %	94.05 %	2.88 %	3.47 %	3.10 %	32
	RF	95.99 %	95.32 %	95.66 %	95.84 %	95.89 %	95.78 %	95.89 %	92.57 %	4.01 %	4.68 %	4.22 %	28
	DT	94.87 %	94.14 %	94.54 %	94.72 %	94.76 %	94.66 %	94.97 %	91.95 %	5.13 %	5.86 %	5.34 %	35
	KNN	93.99 %	93.15 %	93.58 %	93.77 %	93.82 %	93.70 %	93.09 %	90.47 %	6.01 %	6.85 %	6.30 %	42
	GNB	92.71 %	91.83 %	92.27 %	92.49 %	92.52 %	92.45 %	92.11 %	89.23 %	7.29 %	8.17 %	7.55 %	47
	SVM	90.52 %	89.30 %	90.01 %	90.24 %	90.29 %	90.18 %	90.22 %	86.13 %	9.48 %	10.60 %	9.82 %	95
	BLR	87.84 %	87.19 %	87.54 %	87.72 %	87.76 %	87.60 %	87.34 %	83.00 %	12.16 %	12.81 %	12.49 %	60
PSHACK flood	SELDP4-FS	96.42 %	96.81 %	95.11 %	95.25 %	95.26 %	96.08 %	97.44 %	94.69 %	4.74 %	5.19 %	4.92 %	29
	RF	93.97 %	93.14 %	93.59 %	93.78 %	93.79 %	93.56 %	96.52 %	90.84 %	6.21 %	6.86 %	6.44 %	27
	DT	92.83 %	91.97 %	92.39 %	92.58 %	92.59 %	92.37 %	95.61 %	88.53 %	7.41 %	8.02 %	7.63 %	36
	KNN	91.90 %	91.01 %	91.46 %	91.62 %	91.63 %	91.44 %	94.71 %	86.92 %	8.38 %	8.99 %	8.56 %	45
	GNB	90.68 %	89.81 %	90.26 %	90.43 %	90.44 %	90.25 %	92.91 %	85.23 %	9.56 %	10.19 %	9.75 %	49
	SVM	89.02 %	88.17 %	88.64 %	88.82 %	88.83 %	88.63 %	91.24 %	83.02 %	10.99 %	11.83 %	11.37 %	64
	BLR	86.37 %	85.55 %	85.95 %	86.15 %	86.16 %	85.94 %	88.12 %	79.80 %	11.88 %	12.61 %	12.06 %	30

Table 10
Performance comparison of multi-class IoT fragmentation intrusion detection models on CICIoT2023 dataset.

Threat	Model	Accuracy	Precision	F1-score	NPV	Specificity	Recall	AUC	MCC	FPR	FDR	FNR	ADT(ms)
ACK fragmentation	SELDP4-FS	97.85 %	97.21 %	97.52 %	97.62 %	97.68 %	97.40 %	97.35 %	94.98 %	2.15 %	2.79 %	2.60 %	26
	RF	96.74 %	96.12 %	96.42 %	96.52 %	96.59 %	96.30 %	96.54 %	93.68 %	3.26 %	3.88 %	3.70 %	18
	DT	95.62 %	94.91 %	95.24 %	95.32 %	95.41 %	95.10 %	95.12 %	92.37 %	4.38 %	5.09 %	4.90 %	30
	KNN	94.75 %	93.85 %	94.27 %	94.35 %	94.42 %	94.05 %	94.35 %	91.21 %	5.25 %	6.15 %	5.95 %	35
	GNB	93.47 %	92.58 %	92.99 %	93.05 %	93.12 %	92.85 %	93.97 %	89.87 %	6.53 %	7.42 %	7.15 %	42
	SVM	91.82 %	90.69 %	91.26 %	91.34 %	91.42 %	91.12 %	91.42 %	88.02 %	8.18 %	9.31 %	8.88 %	93
	BLR	89.19 %	88.53 %	88.86 %	89.07 %	89.10 %	88.67 %	89.89 %	84.49 %	10.81 %	11.47 %	11.33 %	52
UDP fragmentation	SELDP4-FS	96.65 %	95.01 %	96.33 %	95.47 %	96.48 %	97.31 %	97.22 %	95.76 %	5.53 %	5.99 %	5.69 %	29
	RF	93.21 %	92.59 %	92.93 %	93.06 %	93.07 %	92.89 %	96.03 %	90.38 %	6.94 %	7.41 %	7.11 %	20
	DT	92.07 %	91.38 %	91.73 %	91.88 %	91.89 %	91.69 %	95.10 %	89.01 %	7.96 %	8.31 %	8.31 %	46
	KNN	91.14 %	90.46 %	90.80 %	90.95 %	90.96 %	90.76 %	94.31 %	87.67 %	8.98 %	9.44 %	9.24 %	49
	GNB	89.92 %	89.24 %	89.58 %	89.73 %	89.74 %	89.55 %	92.87 %	86.49 %	10.26 %	10.72 %	10.45 %	58
	SVM	88.26 %	87.61 %	87.94 %	88.09 %	88.10 %	87.91 %	91.11 %	84.55 %	11.91 %	12.39 %	12.09 %	78
	BLR	85.61 %	84.98 %	85.29 %	85.47 %	85.48 %	85.28 %	88.54 %	81.08 %	13.52 %	13.99 %	13.69 %	37

Table 11
Performance comparison of multi-class IoT spoofing intrusion detection models on CICIoT2023 dataset.

Threat	Model	Accuracy	Precision	F1-score	NPV	Specificity	Recall	AUC	MCC	FPR	FDR	FNR	ADT(ms)
DNS Spoofing	SELDP4-FS	95.22 %	94.25 %	94.73 %	95.15 %	95.23 %	95.05 %	95.52 %	94.94 %	4.78 %	5.75 %	4.95 %	33
	RF	94.34 %	93.37 %	93.85 %	95.25 %	94.33 %	94.15 %	94.64 %	89.96 %	5.66 %	6.63 %	5.85 %	38
	DT	93.35 %	92.38 %	92.86 %	93.25 %	93.33 %	93.15 %	93.75 %	88.97 %	6.65 %	7.62 %	6.85 %	42
	KNN	92.36 %	91.39 %	91.87 %	92.25 %	92.33 %	92.15 %	92.86 %	87.98 %	7.64 %	8.61 %	7.85 %	47
	GNB	91.25 %	90.28 %	90.76 %	91.20 %	91.28 %	91.15 %	91.95 %	86.96 %	8.75 %	9.72 %	8.85 %	52
	SVM	90.18 %	89.03 %	89.60 %	90.14 %	90.21 %	90.05 %	90.08 %	85.94 %	9.82 %	10.97 %	9.95 %	98
	BLR	89.02 %	87.95 %	88.48 %	89.00 %	89.08 %	88.95 %	89.22 %	84.93 %	10.98 %	12.05 %	11.05 %	68
ARP Spoofing	SELDP4-FS	97.45 %	96.48 %	96.96 %	97.35 %	97.43 %	97.20 %	97.15 %	94.47 %	2.45 %	3.52 %	2.80 %	20
	RF	96.72 %	95.75 %	96.23 %	98.65 %	97.73 %	96.55 %	96.22 %	93.44 %	3.28 %	4.25 %	3.45 %	26
	DT	96.84 %	94.87 %	96.35 %	95.75 %	95.83 %	97.65 %	95.34 %	92.46 %	4.16 %	5.13 %	4.35 %	30
	KNN	94.85 %	93.88 %	94.36 %	94.75 %	94.83 %	94.65 %	94.45 %	91.47 %	5.15 %	6.12 %	5.35 %	34
	GNB	93.86 %	92.89 %	93.37 %	93.75 %	93.83 %	93.65 %	93.56 %	90.48 %	6.14 %	7.11 %	6.35 %	44
	SVM	92.75 %	91.78 %	92.26 %	92.70 %	92.78 %	92.65 %	92.65 %	89.46 %	7.25 %	8.22 %	7.35 %	64
	BLR	91.68 %	90.53 %	91.10 %	91.64 %	91.71 %	91.55 %	91.78 %	88.44 %	8.32 %	9.47 %	8.45 %	39

Table 12
Performance comparison of multi-class iot reconnaissance intrusion detection models on CICIoT2023 dataset.

Threat	Model	Accuracy	Precision	F1-score	NPV	Specificity	Recall	AUC	MCC	FPR	FDR	FNR	ADT(ms)
Ping sweep	SELDP4-FS	98.15 %	97.40 %	97.77 %	98.05 %	98.13 %	97.90 %	98.25 %	95.68 %	1.85 %	2.60 %	2.10 %	19
	RF	97.42 %	96.78 %	97.10 %	97.45 %	97.53 %	97.35 %	97.32 %	94.65 %	2.58 %	3.22 %	2.65 %	38
	DT	96.54 %	95.90 %	96.22 %	96.55 %	96.63 %	96.45 %	96.44 %	93.67 %	3.46 %	4.10 %	3.55 %	53
	KNN	95.48 %	94.83 %	95.15 %	95.45 %	95.53 %	95.35 %	95.58 %	92.49 %	4.42 %	5.17 %	4.65 %	68
	GNB	94.46 %	93.81 %	94.13 %	94.40 %	94.48 %	94.30 %	94.66 %	91.47 %	5.44 %	6.19 %	5.70 %	62
	SVM	93.35 %	92.70 %	93.02 %	93.30 %	93.38 %	93.20 %	93.75 %	90.46 %	6.55 %	7.30 %	6.80 %	78
	BLR	92.18 %	91.33 %	91.75 %	92.14 %	92.21 %	92.05 %	92.88 %	89.45 %	7.72 %	8.47 %	7.95 %	58
Port scan	SELDP4-FS	96.75 %	95.78 %	96.26 %	96.65 %	96.73 %	96.50 %	96.15 %	93.47 %	3.25 %	4.22 %	3.50 %	27
	RF	96.02 %	95.05 %	95.53 %	95.95 %	96.03 %	95.85 %	96.22 %	92.44 %	3.98 %	4.95 %	4.15 %	44
	DT	95.14 %	94.17 %	94.65 %	95.05 %	95.13 %	94.95 %	95.34 %	91.46 %	4.86 %	5.83 %	5.05 %	48
	KNN	94.15 %	93.18 %	93.66 %	94.05 %	94.13 %	93.95 %	94.45 %	90.47 %	5.85 %	6.82 %	6.05 %	53
	GNB	93.16 %	92.19 %	92.67 %	93.05 %	93.13 %	92.95 %	93.56 %	89.48 %	6.84 %	7.81 %	7.05 %	45
	SVM	92.05 %	91.08 %	91.56 %	92.00 %	92.08 %	91.95 %	92.75 %	88.46 %	7.95 %	8.92 %	8.05 %	69
	BLR	91.08 %	89.93 %	90.50 %	91.04 %	91.11 %	90.95 %	91.88 %	87.44 %	8.92 %	10.07 %	9.05 %	56

algorithm in addressing multi-pronged attacks on IoT. The algorithm's adaptability to diverse network configurations is evident, and the impact of attack complexity on model performance is highlighted in the variations across different attacks. For instance, the accuracy tends to decrease for attacks with higher complexity, emphasizing the need for models capable of handling diverse scenarios. The experimental analysis underscores the efficacy of SELDP4-FS in detecting multi-pronged attacks on IoT, showcasing its adaptability, reliability, and resilience.

Table 9 presents the evaluation of various ML models for multi-class IoT flooding intrusion detection on the CICIoT2023 dataset. The performance results reveal SELDP4-FS as the superior choice. SELDP4-FS consistently achieves the highest Accuracy (96.42 %-97.12 %), F1-score (95.11 %-96.82 %), and AUC (97.44 %-97.72 %) for both RSTFIN and PSHACK floods, demonstrating its effectiveness in accurately classifying attacks. While some models like RF show slightly higher recall in specific cases, SELDP4-FS maintains a strong balance between precision, recall, and low false positive/discovery rates (FPR/FDR: 2.88 %-4.74 %/3.10 %-5.19 %). Although SELDP4-FS has a slightly higher average detection time (ADT: 29-32ms) compared to some models, its superior overall performance in attack classification and minimal errors positions it as a robust intrusion detection system for SDN-IoT networks.

Analyzing the performance of various ML models for multi-class IoT fragmentation attacks reveals SELDP4-FS as the strongest contender. SELDP4-FS demonstrates consistent superiority in key metrics across both ACK and UDP fragmentation attacks. SELDP4-FS achieves the highest Accuracy (97.85 %), F1-score (97.52 %), and AUC (97.35 %) while maintaining low FPR (2.15 %) and FDR (2.79 %) for ACK fragmentation. Compared to RF (the second-best model), SELDP4-FS offers a significant improvement in Accuracy (1.11 %) and F1-score (1.10 %) with minimal trade-offs in other metrics. While the overall accuracy gap between models is slightly larger for UDP fragmentation, SELDP4-FS still exhibits the highest Accuracy (96.65 %) and AUC (97.22 %). Notably, it boasts the best recall (97.31 %) for UDP fragmentation, indicating a superior ability to identify true positive cases (attacks). Although some models (like RF) have a slightly lower FPR, SELDP4-FS maintains a good balance with an FPR of 5.53 %. It's worth mentioning that SELDP4-FS has a slightly higher average detection time (ADT) compared to some models (26-29ms for fragmentation vs. 18-20ms for RF on both attacks). However, considering the significant improvement in core classification metrics and low error rates, this trade-off might be acceptable for achieving robust intrusion detection in SDN-IoT networks. Overall, SELDP4-FS demonstrates itself as a powerful solution for fragment attack detection, offering exceptional accuracy, minimal errors, and a good balance between precision and recall. Table 10 presents a performance comparison of multi-class results for fragmentation attacks in IoT.

The results in Table 11 showcase the performance of various ML models for multi-class IoT spoofing intrusion detection. SELDP4-FS maintains high Accuracy for both attacks (97.45 % for ARP spoofing and 95.22 % for DNS spoofing). However, the 2.23 % difference highlights the added difficulty in accurately classifying DNS spoofing attempts. Similar trends are observed in F1-score (96.96 % vs. 94.73 %) and AUC (97.15 % vs. 95.52 %), indicating a trade-off between precision and recall for DNS spoofing. While all models exhibit lower False Positive Rates (FPR) and False Discovery Rates (FDR) for ARP spoofing compared to DNS spoofing, SELDP4-FS maintains a significant advantage in both categories (FPR: 2.45 % vs. 4.78 %, FDR: 3.52 % vs. 5.75 %). This suggests that SELDP4-FS effectively minimizes misclassifications for both attack types. While SELDP4-FS achieves good results across all attack types, a noticeable performance gap exists between ARP spoofing and DNS spoofing detection. DNS spoofing involves manipulating DNS records to redirect traffic to malicious servers. This complexity can make it challenging to distinguish legitimate DNS requests from spoofed ones, leading to higher classification errors. Additionally, attackers constantly develop new techniques to evade detection. Machine learning models need to be adaptable to capture these evolving patterns, which can be more intricate for DNS spoofing compared to ARP spoofing.

Table 12 presents a comprehensive analysis of multi-class classification results for Reconnaissance attacks on IoT. SELDP4-FS

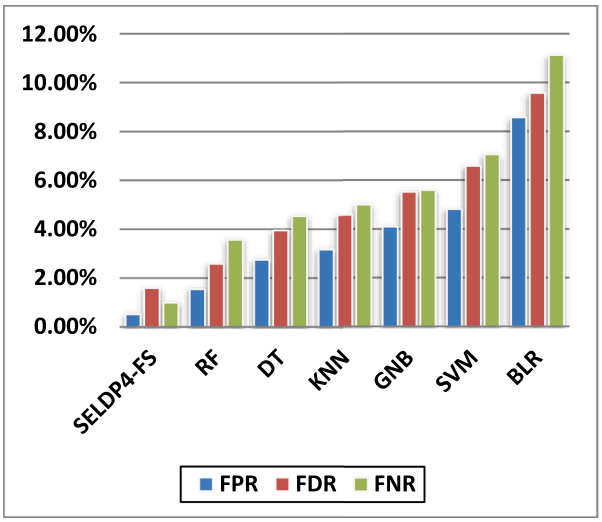


Fig. 12. Error rate analysis for various ML models.

consistently exhibits strong performance across all Reconnaissance attack types, showcasing its effectiveness in detecting Ping sweep, Port scan, and Host discovery. The model demonstrates high accuracy, precision, and recall, indicating its ability to balance between correctly identifying instances of attacks and minimizing false positives. This consistent performance suggests that SELDP4-FS could be a robust choice for detecting multi-pronged Reconnaissance attacks in IoT environments.

However, the comparative analysis also reveals some nuanced strengths and weaknesses among the models. RF and DT models consistently perform well, highlighting the effectiveness of ensemble learning and tree-based approaches for Reconnaissance attack detection. On the other hand, SVM and BLR models exhibit slightly lower performance, indicating potential challenges in handling the complexity of these attacks. The varying metrics across different Reconnaissance attack types suggest that the nature and intricacy of attacks influence the efficacy of detection models. For instance, Ping sweep, being a less complex attack, yields higher performance metrics across all models compared to the more intricate Port scan and Host discovery. The ADT values also highlight the trade-off between detection speed and accuracy, emphasizing the need for a comprehensive understanding of the specific attack scenarios and network environments when deploying detection models. Figs. 12–15 show the system evaluation results from different perspectives.

Table 13 showcases the performance of various ML models in detecting Dictionary Brute Force attacks within the CICIoT2023 dataset. SELDP4-FS stands out as the clear leader, achieving the highest accuracy (96.50 %) and precision (95.23 %). This translates to a model that effectively identifies brute force attempts while minimizing false positives, as evidenced by the high specificity (96.57 %) and low FPR of 3.35 %. This is potentially due to its ensemble learning approach and the dynamic feature selection capabilities through P4 match-action tables. Additionally, the F1-score (95.86 %) and MCC of 92.82 % indicate a balanced performance, meaning the model doesn't favor precision over recall or vice versa. This is crucial for security applications where both identifying attacks and minimizing false alarms are equally important. In comparison, other models exhibit varying degrees of success. RF follows closely with an accuracy of 95.48 % and a precision of 94.21 %. However, its F1-score (94.84 %) and MCC (91.69 %) are slightly lower than SELDP4-FS, suggesting a potential trade-off between precision and recall. DT and KNN show further drops in performance, highlighting the complexity of brute force attacks and the need for robust models. The observed variations can be attributed to several factors. Brute force attacks can take different forms, with some employing simple password dictionaries while others utilize more sophisticated techniques. Additionally, network configuration and the complexity of password policies can influence the difficulty of detecting these attacks.

5.3. *Performance evaluation of single vs. multi-controller architectures*

The MP-GUARD framework's P4-Assisted Cooperative Traffic Monitoring (CTM-P4) module plays a critical role in network optimization. It leverages a multi-controller architecture to significantly enhance network monitoring and overall performance in SD-IoT networks. Table 14 provides a compelling comparison between single-controller and multi-controller architectures across key performance indicators (KPIs), highlighting the substantial benefits offered by MP-GUARD's design choices. Figs. 16–21 present a graphical comparison between the single-controller and multi-controller architectures over six metrics.

The collaboration between controllers plays a key role in boosting efficiency and reducing processing overhead on controllers. This is evident in the Average Overhead Reduction Rate (AORR) metric. Compared to the single-controller scenario, the multi-controller architecture with MCCS achieves significant AORR improvements across all controllers showcased in the table. For instance, Ryu controller experiences a 60 % reduction in overhead (from 45 % to 60 %), while OpenDaylight enjoys a substantial 65 % reduction (from 55 % to 65 %). These reductions in overhead free up valuable controller resources for critical network tasks, leading to improved overall performance.

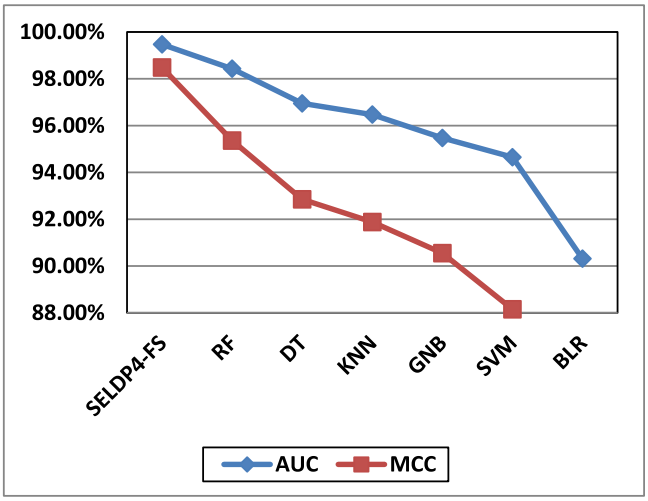


Fig. 13. Correlation and ROC analysis for various ML models.

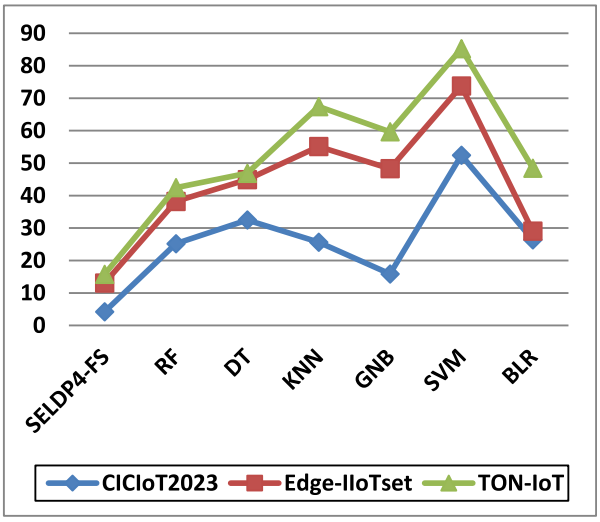


Fig. 14. Evaluating detection time differences across three datasets.

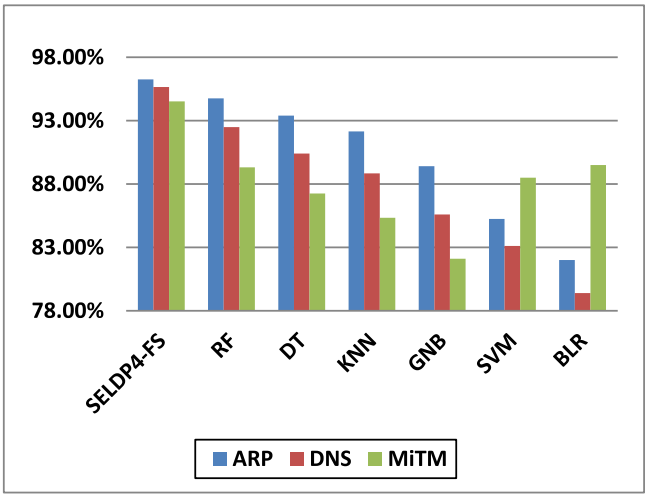


Fig. 15. Accuracy of ML models against stealthy intrusions.

The distributed processing power of the multi-controller architecture offers substantial advantages in scalability compared to the single-controller approach. This is evident in several key metrics. The Average Network Throughput (ANT) for OpenDaylight and Ryu exhibit significant improvements in the multi-controller setup. OpenDaylight's throughput jumps from 120 Mbps in the single-controller scenario to 150 Mbps in the multi-controller configuration, representing a 42 % increase. Similarly, Ryu's throughput increases by 32 %, reaching 145 Mbps in the multi-controller setup. These improvements highlight the multi-controller architecture's ability to handle larger data volumes efficiently within SD-IoT networks.

The multi-controller architecture also demonstrates faster response times through reductions in Average System Latency (ASL). Ryu exhibits a 33 % improvement in latency (18 ms vs. 27 ms), while OpenDaylight achieves a 36 % reduction (16 ms vs. 25 ms) when moving to a multi-controller setup. This suggests improved load balancing and faster reaction to network requests within the SD-IoT network. The multi-controller setup also facilitates more efficient packet handling, as shown by the increase in Packet Processing Speed (PPS) for Ryu. Ryu's PPS increases by 20 % (from 50 Kpps to 60 Kpps) compared to the single-controller scenario, showcasing its faster processing capabilities for incoming packets. OpenDaylight also benefits from the distributed architecture, achieving a PPS of 57 Kpps in the multi-controller setup.

It's important to acknowledge that there are trade-offs to consider when selecting an architecture. While the multi-controller architecture offers significant advantages, it may come at a slight cost of resource utilization. As seen in CPUU and MUT metrics, the multi-controller setup shows slightly higher average resource usage compared to the single-controller approach. However, the increase in resource consumption is relatively modest. For instance, Ryu's CPU utilization increases from 18 % to 20 % in the multi-controller setup, while OpenDaylight's memory utilization goes from 19 % to 22 %.

Table 13
Performance results for dictionary brute force attack for utilized ML models across CICIoT2023 dataset.

Threat	Model	Accuracy	Precision	F1-score	NPV	Specificity	Recall	AUC	MCC	FPR	FDR	FNR	ADT(ms)
Dictionary brute force	SELDP4-FS	96.50 %	95.23 %	95.86 %	96.35 %	96.57 %	96.30 %	96.10 %	92.82 %	3.35 %	4.47 %	3.70 %	27
	RF	95.48 %	94.21 %	94.84 %	95.33 %	95.55 %	95.25 %	95.28 %	91.69 %	4.32 %	5.43 %	4.75 %	49
	DT	94.37 %	93.00 %	93.68 %	94.22 %	94.44 %	94.20 %	94.47 %	89.58 %	5.53 %	6.65 %	6.32 %	34
	KNN	93.36 %	92.09 %	92.72 %	93.21 %	93.43 %	93.20 %	93.56 %	88.47 %	6.54 %	7.66 %	6.80 %	59
	GNB	92.25 %	91.08 %	91.66 %	92.10 %	92.32 %	92.15 %	92.65 %	87.36 %	7.64 %	8.77 %	7.85 %	61
	SVM	91.14 %	89.87 %	90.50 %	91.00 %	91.22 %	90.90 %	91.74 %	86.25 %	8.76 %	9.93 %	9.10 %	86
	BLR	89.82 %	88.55 %	89.18 %	89.70 %	89.92 %	89.70 %	89.32 %	85.13 %	9.98 %	11.15 %	10.30 %	68

Table 14
Performance comparison: single vs. multi-controller architectures.

Architecture	Controller	ANT (Mbps)	ASL (ms)	PDR (%)	PPS (Kpps)	CPUU (%)	MUT (%)	AORR (%)
Single Controller	Ryu	110	27	15	50	18	17	45
	Floodlight	100	30	13	45	23	26	40
	Pox	80	35	11	35	30	17	50
	OpenDaylight	120	25	9	55	25	19	55
Multi-Controller	Ryu	145	18	8	60	20	21	60
	Floodlight	130	22	12	55	30	32	52
	Pox	100	28	7	40	35	24	58
	OpenDaylight	150	16	3	57	27	22	65

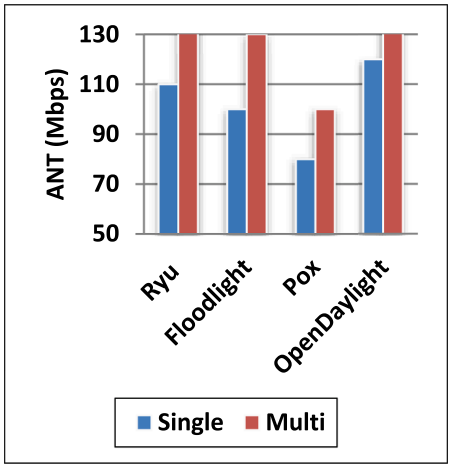


Fig. 16. Comparison of ANT between single and multi-controller architectures.

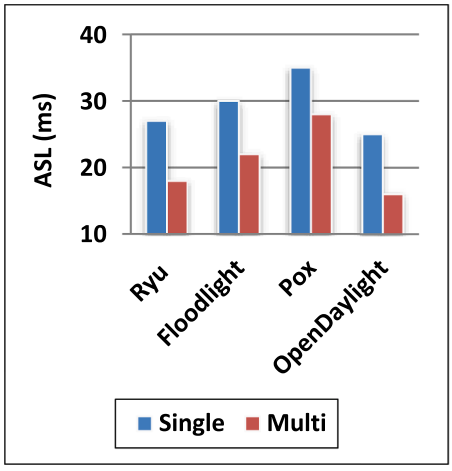


Fig. 17. Comparison of ASL between single and multi-controller architectures.

The choice between single-controller and multi-controller architectures depends on the specific network's requirements. For large-scale, high-performance SD-IoT networks, the multi-controller architecture is the preferred choice due to its superior scalability, evident in its 42 % improvement in throughput for OpenDaylight. Additionally, the multi-controller architecture offers faster response times (36 % reduction in latency for OpenDaylight) and improved packet handling capabilities (20 % increase in PPS for Ryu). These performance improvements come at the cost of slightly higher resource consumption, but the trade-off is worthwhile for large-scale networks.

However, for smaller, resource-constrained networks, the single-controller architecture might suffice. It offers lower resource consumption and potentially simpler management. However, scalability could be limited for future growth. Ultimately, a careful

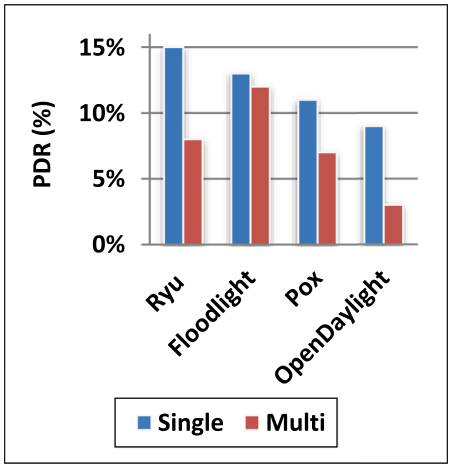


Fig. 18. Comparison of PDR between single and multi-controller architectures.

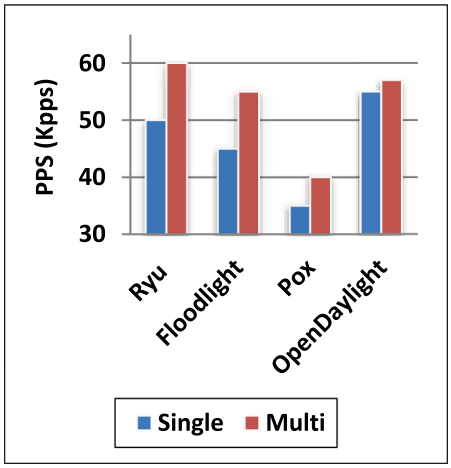


Fig. 19. Comparison of PPS between single and multi-controller architectures.

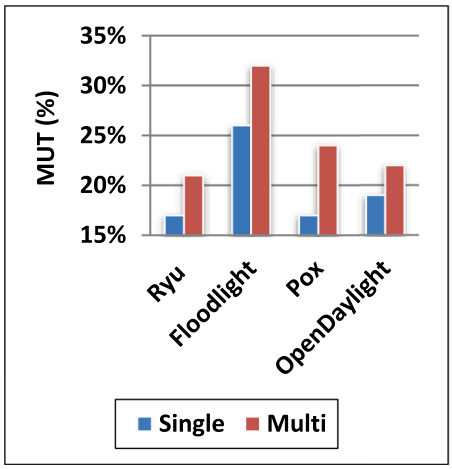


Fig. 20. Comparison of MUT between single and multi-controller architectures.

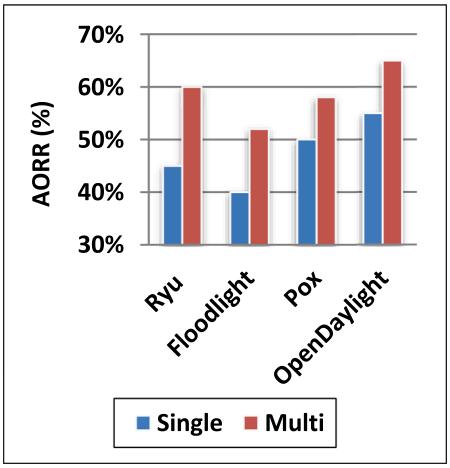


Fig. 21. Comparison of AORR between single and multi-controller architectures.

analysis of network needs and resource availability is crucial for selecting the optimal architecture. MP-GUARD, with its flexible design and data-driven approach exemplified by Table 11, empowers informed decision-making for efficient network management in SD-IoT deployments.

5.4. *Evaluating existing approaches against the proposed framework*

Compared to existing intrusion detection methods, the proposed MP-GUARD framework with its core component, SELDP4-FS, offers a compelling combination of accuracy, adaptability, and robustness for safeguarding SD-IoT networks. MP-GUARD's exceptional 99.32 % detection accuracy surpasses most methods, particularly those limited by single-controller architectures lacking stateful processing. This superiority stems from its innovative design: the ensemble learning approach in SELDP4-FS leverages P4-enabled switches for dynamic feature extraction, resulting in a richer understanding of network traffic and improved attack identification. Furthermore, MP-GUARD exhibits a broader threat coverage compared to methods confined to specific attack types or reliant on outdated datasets. By utilizing comprehensive datasets like CICIoT2023, TON_IoT, and Edge-IIoTset, reflecting real-world IoT traffic patterns, MP-GUARD effectively tackles multi-pronged intrusion attempts. This training on real-world data also contributes to the framework's remarkably low false positive rate (FPR) of 0.49 %, minimizing disruptions caused by misidentifying legitimate traffic as malicious. Finally, the multi-controller architecture of MP-GUARD empowers it with scalability and adaptability. The distributed processing capabilities enable efficient load balancing for handling large-scale SD-IoT networks, while the real-world data training enhances its ability to adapt to diverse network environments. In essence, MP-GUARD addresses the limitations of existing methods by offering a modular architecture, cooperative traffic monitoring, complemented by P4 programmable switches and a synchronized multi-controller design, leading to effective communication, agile attack mitigation, and superior overall security for complex and multi-pronged attacks in SD-IoT networks. Table 15 provides a detailed comparison of the proposed framework against existing

Table 15
Benchmarking performance: proposed MP-GUARD framework vs. existing methods.

Author	Year	Utilized Methodology	Utilized Dataset	Security Task	Architecture	FPR	Accuracy
Awotunde et al. [48]	2021	Feed-forward neural network	UNSW-NB15	Intrusion detection	IIoT	01.10 %	98.90 %
Long et al. [36]	2022	Flow-based (Entropy & SSAE-SVM)	DARPAR	DDoS detection	SDN	04.03 %	98.00 %
Naula [45]	2022	Deep reinforcement learning	Simulated	LR-DDoS detection	SDN	N-A	98.02 %
Omer [53]	2023	Enhanced probabilistic neural network	KDD-CUP 99	Intrusion detection	Conventional	N-A	98.99 %
Khan [47]	2023	Ensemble model using soft voting	CIC-2017	Intrusion detection	IIoT	03.72 %	97.47 %
Zavrak et al. [39]	2023	Flow-based Anomaly Detection (RNNs)	ISCX2012	Intrusion detection	SDN	N-A	99.30 %
Khedr et al. [29]	2023	FMDADM: ML-based multi-layer detection	Edge-IIoTset	DDoS detection	SD-IoT	02.13 %	99.22 %
Singh et al. [40]	2024	Knowledge & Data-driven Ensemble	Generated	Intrusion detection	SD-IoT	N-A	99.50 %
Proposed	2024	SELDP4-FS: Stacked Ensemble Learning with Dynamic P4-Based Feature Selection	TON_IoT, Edge-IIoTset, and CICIoT2023	multi-pronged intrusion attacks	SD-IoT	00.49 %	99.32 %

methods.

6. Limitations and future directions

While MP-GUARD offers a significant step forward in intrusion detection for SD-IoT networks, there are areas for further exploration to address potential limitations and enhance its capabilities. One key challenge lies in balancing the need for real-time feature extraction using P4 programs with the potential for computational overhead on programmable switches. Future research can delve into optimizing P4 programs for efficiency or offloading computationally intensive tasks to dedicated hardware. This would ensure a delicate balance between obtaining rich feature sets, crucial for comprehensive network understanding and maintaining optimal processing power within the network infrastructure.

Scaling MP-GUARD for extensive deployments also requires further investigation. The multi-controller architecture offers a significant advantage, but for large-scale SD-IoT networks, determining the optimal number and placement of controllers becomes critical. Identifying strategies for efficient controller distribution and resource allocation across extensive network deployments is essential. This will ensure effective network coverage and intrusion detection capabilities while maintaining manageability as the network scales.

The ever-evolving threat landscape in the IoT domain necessitates continuous adaptation of MP-GUARD's defense mechanisms. As new attack techniques emerge, the feature set and machine learning models within the framework may require adjustments to maintain effectiveness. Exploring techniques for online model retraining and feature adaptation would allow MP-GUARD to stay ahead of the curve. This could involve incorporating real-time threat intelligence feeds or employing adaptive machine learning algorithms that can learn from new attack patterns, ensuring MP-GUARD remains a robust defense against evolving threats.

While MP-GUARD demonstrates promise in various IoT settings, its performance across diverse environments like smart homes, industrial automation, and connected healthcare systems needs further evaluation. The specific characteristics and security requirements of these environments might necessitate tailoring MP-GUARD's configuration or feature extraction processes. Research in this direction could involve developing domain-specific feature sets or exploring lightweight learning models optimized for resource-constrained IoT devices within these environments.

Finally, integration with emerging technologies like blockchain presents a promising avenue for enhancing security and tamper-proof data storage within MP-GUARD. Research into secure and efficient data-sharing mechanisms within a blockchain-enabled framework is crucial. This could involve exploring secure communication protocols and distributed consensus algorithms suitable for the resource-constrained nature of IoT devices. Integrating blockchain can bolster trust and transparency in data provenance within the intrusion detection process, further strengthening MP-GUARD's security posture.

7. Conclusions

In the ever-expanding realm of the IoT, securing these interconnected devices and networks presents a critical challenge. Traditional intrusion detection methods often struggle with the real-time nature, dynamic traffic patterns, and resource constraints inherent to SD-IoT environments. MP-GUARD emerges as a significant response to this challenge, offering a robust and scalable framework for intrusion detection in these networks.

MP-GUARD's strength lies in its ability to leverage the programmability of SDN. By utilizing P4 programs, the framework extracts features from network traffic in real time, providing a rich and dynamic dataset for analysis. This real-time feature extraction stands in stark contrast to traditional approaches that rely on pre-defined features, potentially missing crucial details about evolving attack patterns.

Furthermore, MP-GUARD employs a multi-controller architecture. This distributed approach not only enhances fault tolerance by ensuring the system remains operational even if a controller fails but also significantly improves scalability. The evaluation demonstrates that the multi-controller architecture allows MP-GUARD to effectively manage large-scale SD-IoT deployments while maintaining efficient intrusion detection capabilities. This scalability is crucial as the number and complexity of interconnected devices within IoT networks continue to grow.

The effectiveness of MP-GUARD is further underscored by its exceptional performance in terms of detection accuracy. The ensemble learning technique (SELDP4-FS) utilizes the P4-extracted features to achieve superior accuracy in attack identification compared to traditional approaches. This superior accuracy translates to a more reliable and secure network environment, ensuring that legitimate traffic flows freely while malicious activity is effectively identified and thwarted.

Beyond accuracy, MP-GUARD prioritizes efficiency. By performing intrusion detection at the data plane level, closer to the raw network traffic, MP-GUARD minimizes the processing burden on controllers. This reduction in controller load translates to improved overall system responsiveness. In resource-constrained SD-IoT environments, efficient resource utilization is paramount, and MP-GUARD's design ensures that network resources are used effectively for both intrusion detection and core network operations.

The MASL metric adds another layer of sophistication to MP-GUARD. This metric provides valuable insights into the stability of the machine learning models under varying traffic conditions. Network traffic can fluctuate significantly, and ensuring that the intrusion detection system remains reliable even during periods of peak activity or sudden traffic surges is crucial. The MASL metric allows for proactive monitoring and adjustments to maintain optimal performance, guaranteeing reliable attack detection regardless of network traffic patterns.

In conclusion, MP-GUARD presents a significant contribution to the field of IoT security. Its modular design, real-time feature extraction using P4, distributed processing architecture with a multi-controller approach, and focus on efficiency and stability position

it as a powerful tool for securing future SD-IoT deployments. By continuously evolving to address potential limitations and explore future directions, MP-GUARD can ensure robust and adaptable intrusion detection capabilities in the face of the ever-growing threat landscape within IoT networks.

CRediT authorship contribution statement

Ameer El-Sayed: Writing – original draft, Conceptualization, Methodology, Investigation, Software, Validation. **Wael Said:** Conceptualization, Methodology, Writing – review & editing. **Amr Tolba:** Conceptualization, Methodology, Investigation, Writing – review & editing. **Yasser Alginahi:** Conceptualization, Methodology, Writing – review & editing. **Ahmed A. Toony:** Conceptualization, Methodology, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgement

This work was funded by the Researchers Supporting Project Number (RSPD2024R681), King Saud University, Riyadh, Saudi Arabia.

References

[1] Alani MM. An explainable efficient flow-based Industrial IoT intrusion detection system. Comput Electr Eng 2023;108:108732.
[2] Kumar A, Abhishek K, Ghalib MR, Shankar A, Cheng X. Intrusion detection and prevention system for an IoT environment. Digit Commun Netw 2022;8:540–51.
[3] Nour AA, Mehbodniya A, Webber JL, Bostani A, Shah B, Ergashevich BZ, et al. Optimizing intrusion detection in industrial cyber-physical systems through transfer learning approaches. Comput Electr Eng 2023;111:108929.
[4] Cao Y, Wang Z, Ding H, Zhang J, Li B. An intrusion detection system based on stacked ensemble learning for IoT network. Comput Electr Eng 2023;110:108836.
[5] Kasongo SM. A deep learning technique for intrusion detection system using a recurrent neural networks based framework. Comput Commun 2023;199:113–25.
[6] Majidian Z, TaghipourEivazi S, Arasteh B, Babaie S. An intrusion detection method to detect denial of service attacks using error-correcting output codes and adaptive neuro-fuzzy inference. Comput Electr Eng 2023;106:108600.
[7] Ravi V, Chaganti R, Alazab M. Recurrent deep learning-based feature fusion ensemble meta-classifier approach for intelligent network intrusion detection system. Comput Electr Eng 2022;102:108156.
[8] Sarhan M, Lo WW, Layeghy S, Portmann M. HBFL: a hierarchical blockchain-based federated learning framework for collaborative IoT intrusion detection. Comput Electr Eng 2022;103:108379.
[9] Niakanlahiji A, Orlowski S, Vahid A, Jafarian JH. Toward practical defense against traffic analysis attacks on encrypted DNS traffic. Comput Secur 2023;124:103001.
[10] Simon J, Kapileswar N, Polasi PK, Elaveini MA. Hybrid intrusion detection system for wireless IoT networks using deep learning algorithm. Comput Electr Eng 2022;102:108190.
[11] Webber JL, Arafa A, Mehbodniya A, Karupusamy S, Shah B, Dahiya AK, et al. An efficient intrusion detection framework for mitigating blackhole and sinkhole attacks in healthcare wireless sensor networks. Comput Electr Eng 2023;111:108964.
[12] Sanlı M. Detection and mitigation of denial of service attacks in internet of things networks. Arab J Sci Eng 2024:1–11.
[13] Habib AA, Hasan MK, Alkhayyat A, Islam S, Sharma R, Alkwai LM. False data injection attack in smart grid cyber physical system: Issues, challenges, and future direction. Comput Electr Eng 2023;107:108638.
[14] Faramondi L, Flammini F, Guarino S, Setola R. A hybrid behavior-and Bayesian network-based framework for cyber–physical anomaly detection. Comput Electr Eng 2023;112:108988.
[15] Kumar S, Kumar A. Image-based malware detection based on convolution neural network with autoencoder in Industrial Internet of Things using Software Defined Networking Honeypot. Eng Appl Artif Intell 2024;133:108374.
[16] Shaji NS, Muthalagu R, Pawar PM. SD-IIDS: intelligent intrusion detection system for software-defined networks. Multimed Tools Appl 2024;83:11077–109.
[17] Li DC, Tu HH, Chou LD. Cross-layer detection and defence mechanism against DDoS and DRDoS attacks in software-defined networks using P4 switches. Comput Electr Eng 2024;118:109307.
[18] Sarma B, Kumar R, Tuithung T. Machine learning enabled network and task management in SDN based Fog architecture. Comput Electr Eng 2023;108:108705.
[19] Aslam N, Srivastava S, Gore M. DDoS SourceTracer: an intelligent application for DDoS attack mitigation in SDN. Comput Electr Eng 2024;117:109282.
[20] Kim J, Kim Y, Yegneswaran V, Porras P, Shin S, Park T. Extended data plane architecture for in-network security services in software-defined networks. Comput Secur 2023;124:102976.
[21] Kaur S, Kumar K, Aggarwal N. A review on P4-Programmable data planes: architecture, research efforts, and future directions. Comput Commun 2021;170:109–29.
[22] Chen L, Wang Z, Huo R, Huang T. An adversarial DBN-LSTM method for detecting and defending against DDoS attacks in SDN environments. Algorithms 2023;16:197.
[23] Hauser F, Häberle M, Merling D, Lindner S, Gurevich V, Zeiger F, et al. A survey on data plane programming with p4: fundamentals, advances, and applied research. J Netw Comput Appl 2023;212:103561.
[24] Liu Z, Wang Y, Feng F, Liu Y, Li Z, Shan Y. A DDoS detection method based on feature engineering and machine learning in software-defined networks. Sensors 2023;23:6176.
[25] Lee AYP, Wang MIC, Hung CH, Wen CHP. PS-IPS: deploying intrusion prevention system with machine learning on programmable switch. Future Gener Comput Syst 2024;152:333–42.
[26] Bashendy M, Tantawy A, Erradi A. Intrusion response systems for cyber-physical systems: a comprehensive survey. Comput Secur 2023;124:102984.

[27] Hernandez-Jaimes ML, Martinez-Cruz A, Ramírez-Gutiérrez KA, Feregrino-Uribe C. Artificial intelligence for IoMT security: a review of intrusion detection systems, attacks, datasets and Cloud-Fog-Edge architectures. Internet Things 2023:100887.
[28] Mohammadian H, Ghorbani AA, Lashkari AH. A gradient-based approach for adversarial attack on deep learning-based network intrusion detection systems. Appl Soft Comput 2023;137:110173.
[29] Khedr WI, Gouda AE, Mohamed ER. P4-HLDMC: A novel framework for DDoS and ARP attack detection and mitigation in SD-IoT networks using machine learning, stateful P4, and distributed multi-controller architecture. Mathematics 2023;11:3552.
[30] Lazzarini R, Tianfield H, Charissis V. A stacking ensemble of deep learning models for IoT intrusion detection. Knowl Based Syst 2023;279:110941.
[31] Aldarwbi MY, Lashkari AH, Ghorbani AA. The sound of intrusion: a novel network intrusion detection system. Comput Electr Eng 2022;104:108455.
[32] Zhang J, Peter JD, Shankar A, Viriyasitavat W. Public cloud networks oriented deep neural networks for effective intrusion detection in online music education. Comput Electr Eng 2024;115:109095.
[33] Zhu J, Liu X. An integrated intrusion detection framework based on subspace clustering and ensemble learning. Comput Electr Eng 2024;115:109113.
[34] Mulyanto M, Leu JS, Faisal M, Yunanto W. Weight embedding autoencoder as feature representation learning in an intrusion detection systems. Comput Electr Eng 2023;111:108949.
[35] Habiba M, Islam MR, Muyeen S, Ali AS. Edge intelligence for network intrusion prevention in IoT ecosystem. Comput Electr Eng 2023;108:108727.
[36] Long Z, Jinsong W. A hybrid method of entropy and SSAE-SVM based DDoS detection and mitigation mechanism in SDN. Comput Secur 2022;115:102604.
[37] Zhang N, Jaafar F, Malik Y. Low-rate DoS attack detection using PSD based entropy and machine learning. In: Proceedings of the 2019 6th IEEE international conference on cyber security and cloud computing (CSCloud)/2019 5th IEEE international conference on edge computing and scalable cloud (EdgeCom),; 2019. p. 59–62. https://doi.org/10.1109/CSCloud/EdgeCom.2019.00020.
[38] Xie J, Yu FR, Huang T, Xie R, Liu J, Wang C, et al. A survey of machine learning techniques applied to software defined networking (SDN): Research issues and challenges. IEEE Commun Surv Tutor 2018;21:393–430. https://doi.org/10.1109/COMST.2018.2866942.
[39] Zavrak S, Iskefiyeli M. Flow-based intrusion detection on software-defined networks: a multivariate time series anomaly detection approach. Neural Comput Appl 2023;35:12175–93.
[40] Singh A, Chouhan PK, Aujla GS. SecureFlow: Knowledge and data-driven ensemble for intrusion detection and dynamic rule configuration in software-defined IoT environment. Ad Hoc Netw 2024;156:103404.
[41] Ravi N, Shalinie SM. Learning-driven detection and mitigation of DDoS attack in IoT via SDN-cloud architecture. IEEe Internet Things J 2020;7:3559–70. https://doi.org/10.1109/JIOT.2020.2973176.
[42] Yin D, Zhang L, Yang K. A DDoS attack detection and mitigation with software-defined Internet of Things framework. IEEe Access 2018;6:24694–705. https://doi.org/10.1109/ACCESS.2018.2831284.
[43] Yousuf O, Mir RN. DDoS attack detection in Internet of Things using recurrent neural network. Comput Electr Eng 2022;101:108034. https://doi.org/10.1016/j.compeleceng.2022.108034.
[44] Tang TA, Mhamdi L, McLernon D, Zaidi SAR, Ghogho M. Deep learning approach for network intrusion detection in software defined networking. In: Proceedings of the 2016 international conference on wireless networks and mobile communications (WINCOM); 2016. p. 258–63. https://doi.org/10.1109/WINCOM.2016.7777224.
[45] Yungaicela-Naula NM, Vargas-Rosales C, Pérez-Díaz JA, Carrera DF. A flexible SDN-based framework for slow-rate DDoS attack mitigation by using deep reinforcement learning. J Netw Comput Appl 2022;205:103444.
[46] de Assis MV, Carvalho LF, Rodrigues JJ, Lloret J, Proença Jr ML. Near real-time security system applied to SDN environments in IoT networks using convolutional neural network. Comput Electr Eng 2020;86:106738. https://doi.org/10.1016/j.compeleceng.2020.106738.
[47] Khan MA, Iqbal N, Jamil H, Kim DH. An optimized ensemble prediction model using AutoML based on soft voting classifier for network intrusion detection. J Netw Comput Appl 2023;212:103560.
[48] Awotunde JB, Chakraborty C, Adeniyi AE. Intrusion detection in industrial internet of things network-based on deep learning model with rule-based feature selection. Wirel Commun Mob Comput 2021;2021. https://doi.org/10.1155/2021/7154587.
[49] Aslam M, Ye D, Tariq A, Asad M, Hanif M, Ndzi D, et al. Adaptive machine learning based distributed denial-of-services attacks detection and mitigation system for SDN-enabled IoT. Sensors 2022;22:2697.
[50] Alsaedi A, Moustafa N, Tari Z, Mahmood A, Anwar A. TON_IoT telemetry dataset: a new generation dataset of IoT and IIoT for data-driven intrusion detection systems. IEEe Access 2020;8:165130–50.
[51] Ferrag MA, Ndhlovu M, Tihanyi N, Cordeiro LC, Debbah M, Lestable T, et al. Revolutionizing cyber threat detection with large language models: a privacy-preserving BERT-based lightweight model for IoT/IIoT devices. IEEe Access 2024.
[52] Neto ECP, Dadkhah S, Ferreira R, Zohourian A, Lu R, Ghorbani AA. CICIoT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment. Sensors 2023;23:5941.
[53] Omer N, Samak AH, Taloba AI, Abd El-Aziz RM. A novel optimized probabilistic neural network approach for intrusion detection and categorization. Alex Eng J 2023;72:351–61.